

APLICACIÓN DE MODELOS DE INTELIGENCIA ARTIFICIAL PARA LA CLASIFICACIÓN Y EL ANÁLISIS DE CATÁLOGOS DE TIENDAS

Sistema Multi-Agente de Recomendación de Moda

Presentado por:

Gustavo Daniel Gago Hernández

Programa Académico:

Diploma en Ciencia de Datos e Inteligencia Artificial Aplicada

Instituciones Colaboradoras:

Universidad Tecnológica, UTEC (Uruguay)
Tecnológico de Monterrey (México)

Tutor Académico:

Lic. Martín Fernández

Fecha de Presentación:

Diciembre 2025

Grupo 12 - Trabajo Final de Diploma
Montevideo, Uruguay

Abstract

Este trabajo presenta un sistema inteligente de recomendación de moda basado en embeddings multimodales y arquitectura multiagente, aplicado a datos reales de tiendas uruguayas proporcionados por la startup *Styles Designer* [8]. El total de datos proporcionados fue de 16660 artículos.

El desarrollo del sistema incluyó una etapa de limpieza de datos, ya que los mismos fueron obtenidos mediante técnicas de scrapping y por tal motivo se estandarizó la categoría y el género de las prendas.

Se generaron embeddings bilingües (español/inglés) de 1024 dimensiones (512D por idioma) utilizando OpenFashion CLIP [2], concatenando representaciones multilingües para enriquecer la capacidad predictiva. Este proceso permitió entrenar modelos con capacidad real de generalización sobre 596 productos, tras el filtrado de categorías con menos de cinco muestras.

El sistema de recomendación integra tres componentes principales: (1) clasificación supervisada con CatBoost [7] aplicando el balanceo de clases (*class weights*) para mitigar el desbalanceo, alcanzando F1-scores de $84.01\% \pm 3.26\%$ en categorización de 23 clases y $91.19\% \pm 5.84\%$ en clasificación de 6 géneros mediante validación cruzada estratificada de 5 folds; (2) búsqueda vectorial híbrida implementada con PostgreSQL + pgvector; y (3) sistema multiagente que integra GPT-4 Vision (Azure OpenAI) [6], clasificación CatBoost, y embeddings CLIP para búsqueda semántica.

La arquitectura orquesta 7 agentes especializados mediante un coordinador inteligente basado en Azure OpenAI GPT-4 [6]. La innovación clave reside en un **sistema de decisión híbrido** que integra tres niveles de predicción jerárquicos: (i) clasificación CatBoost (gradient boosting) [7] como método primario, (ii) análisis semántico con embeddings OpenFashion CLIP como fallback inteligente y (iii) inferencia por votación mayoritaria desde los $k = 5$ productos más similares mediante búsqueda vectorial (pgvector) cuando ambos métodos anteriores presentan confianza inferior al 50 %, garantizando resultados útiles incluso en consultas ambiguas.

Palabras clave: sistemas de recomendación, embeddings multimodales, aprendizaje automático, búsqueda semántica, moda, sistemas multiagente.

Agradecimientos

Este trabajo de diploma representa el cierre de una etapa formativa que no habría sido posible sin el apoyo y la colaboración de diversas personas e instituciones a quienes deseo expresar mi más sincero agradecimiento.

En primer lugar, agradezco a mi tutor, **Lic. Martín Fernández**, por su orientación académica, su paciencia y sus valiosas sugerencias durante todo el desarrollo de este proyecto. Sus aportes fueron fundamentales para estructurar y llevar adelante este trabajo. Sobre todo, agradezco su visión clara sobre el alcance del proyecto: propuso desafíos que inicialmente parecían inalcanzables, pero que, al final del camino, logré cumplir en tiempo y forma. Su confianza en mi capacidad fue un motor esencial.

A la **Universidad Tecnológica de Uruguay (UTEC)** y al **Tecnológico de Monterrey (México)**, instituciones que brindaron los recursos académicos, tecnológicos y el ambiente propicio para un aprendizaje de excelencia durante toda la diplomatura.

A la startup **Styles Designer**, por permitir el acceso a su base de datos real de productos uruguayos, un recurso indispensable para desarrollar este trabajo aplicado. Espero que parte del contenido de este documento pueda serles de utilidad.

A los profesores de la diplomatura, por su dedicación, compromiso y excelencia académica demostrada en cada etapa del programa. Su paciencia y profesionalismo fueron fundamentales para mi formación técnica.

A mi familia y amigos, los que están y los que ya no, por su comprensión y apoyo incondicional. Especialmente a Alison y a nuestra hija Belén, cuyo amor y apoyo cotidianos son pilares fundamentales para que esto haya cumplido su ciclo.

Finalmente, agradezco a la comunidad de código abierto y a los investigadores cuyos trabajos previos sirvieron como base teórica y referencia metodológica para este proyecto. Sin su contribución al conocimiento colectivo, este trabajo no habría sido posible.

Gustavo Daniel Gago Hernández
Diciembre, 2025

Índice general

Agradecimientos	2
Glosario	6
1. Introducción	10
1.1. Presentación del problema objetivo	10
1.2. Justificación	10
1.3. Objetivos	10
2. Revisión de la Literatura	11
2.1. Análisis de investigaciones previas	11
3. Marco Teórico	13
3.1. Conceptos teóricos	13
3.2. Cálculo de Similitud con vectores en pgvector	13
3.2.1. Operadores de Distancia Coseno en pgvector	14
3.2.2. Conversión de Distancia a Similitud	14
3.2.3. Interpretación de Resultados	14
3.2.4. Implementación SQL con similitud coseno ($\leq$) en pgvector	14
3.2.5. Similitud Combinada	15
3.2.6. Rangos de Interpretación	15
3.3. Operadores de Distancia Euclidiana (L2) en pgvector	15
3.3.1. Implementación SQL de la distancia Euclidiana	16
3.4. Optimización de Búsqueda Vectorial: Índices HNSW	16
3.5. Embeddings	18
3.6. RAG (Retrieval Augmented Generation)	19
4. Metodología	21
4.1. Diseño del estudio	21
4.1.1. Fases del Diseño Metodológico	21
4.2. Recopilación de datos	23
4.3. Preprocesamiento de datos	23
4.3.1. Análisis de categorías y géneros en las tiendas	23
4.3.2. definición de colores por las tiendas	25
4.4. Proceso de limpieza y estandarización de datos	26
4.4.1. Recopilación y Consolidación de Datos	26
4.4.2. Clasificación Automática con LLM	27
4.4.3. Estandarización de Categorías Globales	27
4.4.4. Conversión de imágenes a Base64 para almacenamiento y procesamiento	28
4.4.5. Uso del modelo facebook/bart-large-mnli	29
4.4.6. Uso del modelo OpenAI en Azure	29
4.4.7. Uso de Open AI para determinar las categorías y el género de las prendas	30
4.4.8. Preparación de base de datos para almacenar la información obtenida	31
4.4.9. Preparando los textos para generar los embeddings	32
4.5. Generación de embeddings de texto e imágenes	32

4.6.	Pruebas de búsquedas por medio de la distancia coseno en la base de datos . . .	33
4.6.1.	Decisión de Arquitectura: Sistema Híbrido CatBoost + LLM	37
4.7.	Generación de Mapping en categoría y en género con CatBoost	37
4.8.	Evaluación del <i>Image Embedding</i> con Catboost	39
4.8.1.	Problema de Ruido Contextual	39
4.8.2.	Decisión Metodológica	39
4.9.	Entrenamientos por peso en datos desbalanceados	39
4.9.1.	Categoría	39
4.9.2.	Género	42
4.9.3.	Resumen de los entrenamientos	44
4.10.	Métricas de Robustez y Generalización	44
4.11.	Interfaz de usuario y complementos	45
4.11.1.	Sistema de inicio de sesión	46
4.11.2.	Sidebar de configuración y ajustes	46
4.11.3.	Menú de navegación del sistema	47
4.12.	Diagrama macro del sistema	48
4.13.	Stack tecnológico	49
4.14.	Fundamentos teóricos de sistemas multi-agente	50
4.14.1.	Clasificación de agentes	50
4.15.	Arquitectura de coordinación multi-agente con LangChain	50
4.15.1.	Componentes de LangChain utilizados	50
4.15.2.	Justificación del uso limitado de LangChain	51
4.16.	Arquitectura propia de coordinación	51
4.16.1.	Flujos de ejecución: dinámico vs estático	55
4.16.2.	Comparación con framework estándar de LangChain	55
4.16.3.	Resultados de la arquitectura híbrida	56
4.17.	Gestión de Memoria y Contexto Conversacional	56
4.17.1.	Estructuras de Datos en Memoria	56
4.17.2.	Optimizaciones de Rendimiento	57
4.17.3.	Ventanas de Contexto por Agente	57
4.17.4.	Persistencia en PostgreSQL	58
4.17.5.	Comparativa Memoria Híbrida	58
4.18.	Agentes Especializados	58
4.18.1.	Coordinador Inteligente	58
4.18.2.	Analizador de Consulta	59
4.18.3.	Analizador de Imágenes	59
4.18.4.	Clasificador CatBoost	60
4.18.5.	Especialista en Outfits	60
4.18.6.	Refinador de Consulta	60
4.18.7.	Buscador de Embeddings	61
4.18.8.	Presentador de Resultados	63
4.18.9.	Resumen de Arquitectura	64
4.18.10.	Sistema Híbrido de Corrección Ortográfica	64
4.19.	Consideraciones Éticas	66
4.19.1.	Recopilación de Datos mediante Web Scraping	66
4.19.2.	Protección de Datos Personales	66
4.19.3.	Privacidad y Uso de APIs Externas	66
4.19.4.	Sesgos Algorítmicos y Equidad	67
4.19.5.	Transparencia y Explicabilidad	67
4.19.6.	Responsabilidad en el Despliegue	68
5.	Análisis y Resultados	69
5.1.	Resultados obtenidos por la api	69

6. Discusión	74
6.1. Interpretación de los resultados	74
6.2. Comparación con la literatura	74
6.3. Limitaciones	74
6.4. Recomendaciones	76
7. Conclusiones	78
Anexos	81

Glosario de Términos Técnicos

Términos de Inteligencia Artificial y Machine Learning

Embeddings Representaciones vectoriales de datos (texto, imagen, audio) en un espacio de alta dimensión. En este proyecto se utilizan embeddings de 512 dimensiones generados por OpenFashion CLIP.

OpenFashion CLIP

Modelo multimodal preentrenado especializado en moda que genera embeddings de 512 dimensiones para texto e imágenes de prendas de vestir.

CatBoost Algoritmo de *Gradient Boosting* desarrollado por Yandex, utilizado en este proyecto para la clasificación de categorías (23 clases) y géneros (6 clases) de prendas.

Data Leakage Inclusión inadvertida de las variables objetivo en las características de entrenamiento, provocando métricas artificialmente altas sin capacidad de generalización real.

LLM (Large Language Model)

Modelos de lenguaje de gran escala como GPT-4 o GPT-4o-mini, capaces de comprender y generar texto en lenguaje natural.

RAG (Retrieval Augmented Generation)

Arquitectura que combina un sistema de recuperación de información con un modelo generativo (LLM) para producir respuestas basadas en contexto específico.

Búsqueda vectorial con votación mayoritaria

Estrategia de *fallback* que infiere categoría/género mediante consenso de los $k = 5$ productos visualmente más similares.

Distancia Coseno

Métrica de similitud que mide el ángulo entre dos vectores. Valores cercanos a 1 indican alta similitud; cercanos a 0, baja similitud.

Distancia Euclidiana (L2)

Métrica que mide la distancia geométrica directa entre dos puntos en un espacio vectorial, equivalente a la norma L_2 del vector diferencial.

Cross-Validation (Validación Cruzada)

Técnica estadística para evaluar la robustez de un modelo dividiéndolo en múltiples particiones (folds) y midiendo su rendimiento promedio.

F1-Score

Métrica de evaluación que combina precisión y recall mediante su media armónica. Rango: $[0, 1]$, donde 1 indica predicción perfecta.

Overfitting (Sobreajuste)

Fenómeno donde un modelo aprende ruido específico del conjunto de entrenamiento, perdiendo capacidad de generalización en datos nuevos.

Early Stopping

Técnica de regularización que detiene el entrenamiento cuando no hay mejora en el conjunto de validación durante N iteraciones consecutivas.

Términos de Bases de Datos y Búsqueda Vectorial

pgvector Extensión de PostgreSQL que permite almacenar vectores y realizar búsquedas de similitud mediante operadores especializados ($\leq$, $\leq\#$).

HNSW (Hierarchical Navigable Small World)

Estructura de índice jerárquica para búsqueda aproximada de vecinos más cercanos (ANN) con complejidad sublineal $O(\log n)$.

Operador $\leq$

Operador de pgvector que calcula la distancia coseno estándar en el rango $[0, 2]$. Se convierte a similitud mediante $\text{similitud} = 1 - \frac{\text{distancia}}{2}$.

Operador $\leq\#$

Operador de pgvector que calcula la distancia coseno negativa en el rango $[-1, 1]$.

Búsqueda Híbrida

Estrategia que combina múltiples fuentes de similitud (texto español, texto inglés, imagen) mediante fusión ponderada.

Términos del Sistema Multiagente

Coordinador Agente orquestador que gestiona la comunicación entre los **7 agentes especializados** del sistema y determina qué agentes invocar según el contexto.

AnalizadorConsulta

Agente LLM que extrae atributos estructurados (color, categoría, género, estilo) de las consultas en lenguaje natural del usuario.

ImageAnalyzer

Agente basado en GPT-4 Vision que genera descripciones textuales detalladas de imágenes de prendas cargadas por el usuario.

EspecialistaOutfits

Agente LLM especializado en generar combinaciones de prendas según ocasiones específicas (casual, formal, deportivo).

ClasificadorCatBoost

Agente que encapsula los modelos de CatBoost entrenados para predecir categoría y género con scores de confianza.

PresentadorResultados

Agente LLM que formatea las respuestas al usuario en lenguaje natural, integrando los productos encontrados y el contexto conversacional.

RefinadorConsulta

Agente que detecta consultas ambiguas y solicita aclaraciones al usuario para mejorar la precisión de las búsquedas.

Términos de Interfaz y Arquitectura

Streamlit Framework de Python para desarrollo rápido de aplicaciones web interactivas, utilizado para el frontend del sistema.

LangChain Framework de Python para orquestación de agentes LLM, gestión de prompts, memoria conversacional y chains de procesamiento.

Base64	Esquema de codificación que convierte datos binarios (imágenes) en cadenas de texto ASCII para transmisión y almacenamiento.
Prompt	Instrucción en lenguaje natural que define el comportamiento y especialización de un agente LLM.
Temperatura (LLM)	Parámetro que controla la creatividad/determinismo de un LLM. Rango: [0, 1], donde 0 es determinista y 1 es creativo.
JWT (JSON Web Token)	Estándar de token de autenticación que codifica información del usuario en formato JSON firmado, utilizado para gestión de sesiones.
Webhook	Mecanismo de comunicación HTTP que permite notificaciones en tiempo real entre sistemas mediante callbacks automáticos.

Acrónimos Utilizados

API	<i>Application Programming Interface</i> Interfaz de programación de aplicaciones.
BD	Base de datos.
CLIP	<i>Contrastive Language-Image Pretraining</i> Modelo multimodal de OpenAI.
CNN	<i>Convolutional Neural Network</i> Red neuronal convolucional.
CSV	<i>Comma-Separated Values</i> Archivo de valores separados por comas.
DF	<i>DataFrame</i> Estructura de datos tabular de pandas.
ES	Español (idioma).
EN	Inglés (idioma).
GPU	<i>Graphics Processing Unit</i> Unidad de procesamiento gráfico.
HTTP	<i>HyperText Transfer Protocol</i> Protocolo de transferencia de hipertexto.
JSON	<i>JavaScript Object Notation</i> Notación de objetos de JavaScript.
ML	<i>Machine Learning</i> Aprendizaje automático.
NLP	<i>Natural Language Processing</i> Procesamiento de lenguaje natural.
SQL	<i>Structured Query Language</i> Lenguaje de consulta estructurado.
UX	<i>User Experience</i> Experiencia de usuario.

Métricas de Evaluación

Accuracy (Exactitud)

Proporción de predicciones correctas sobre el total. Fórmula:

$$\frac{TP + TN}{TP + TN + FP + FN}.$$

Precision (Precisión)

Proporción de predicciones positivas correctas sobre el total de predicciones positivas. Fórmula:

$$\frac{TP}{TP + FP}.$$

Recall (Exhaustividad)

Proporción de casos positivos correctamente identificados sobre el total de positivos reales. Fórmula:

$$\frac{TP}{TP + FN}.$$

Similitud Medida normalizada en $[0, 1]$ que indica qué tan cercanos son dos vectores. En este proyecto: $\text{similitud} = 1 - \frac{\text{distancia_coseno}}{2}$.

Nota: Este glosario cubre los términos técnicos principales utilizados en el documento. Para definiciones más detalladas, consultar las secciones correspondientes en el texto principal.

Capítulo 1

Introducción

1.1 Presentación del problema objetivo

En el ámbito comercial, principalmente en las tiendas de ropa, nos encontramos con un catálogo de prendas sumamente diverso, lo cual dificulta a los clientes encontrar exactamente lo que buscan, obligándolos a gastar tiempo navegando entre múltiples comercios. Este trabajo se enfoca en el estudio de los datos provistos por la empresa Styles Designer [8], conformados por 4 archivos CSV (`product_data.csv`, `categories.csv`, `products.csv` y `shops.csv`) que presentan diversidad de criterios entre tiendas, generando datos heterogéneos. El objetivo es implementar: (1) un sistema de búsqueda multimodal cross-tienda (texto + imágenes) capaz de encontrar prendas mediante interacción en lenguaje natural, y (2) un módulo de recomendación para facilitar la construcción de outfits coherentes utilizando el catálogo completo disponible. Para cumplir ambos objetivos se plantea el uso de técnicas de IA en las diferentes etapas del proceso.

1.2 Justificación

Estos archivos serán procesados generando un único *dataframe* (DF) para estandarizar las categorías, géneros, colores, financiaciones, etc. Esto facilita significativamente el estudio posterior para poder identificar prendas y tiendas de forma más efectiva, para reducir el tiempo de proceso, costos y búsqueda por parte del generador de outfits por cada nuevo *scraping* que se les haga a las URLs de las tiendas.

1.3 Objetivos

- Objetivo general: Desarrollar un recomendador en base a los catálogos de ropa de 35 tiendas de ropa en Uruguay con un dataset de 50000 filas.
- Objetivos específicos:
 - Data Analytics
 - Procesamiento y estandarización de datos
 - Colecciones
 - Colores
 - Tipos
 - Tallas
 - Entrenar modelos de la supervisados y no supervisados
 - Uso de LLM y ML
 - Agentes y Multiagentes

Capítulo 2

Revisión de la Literatura

2.1 Análisis de investigaciones previas

Entre las investigaciones encontradas previamente para este trabajo, se identificaron varios trabajos relacionados con la Universidad de Cornell¹ y una publicación de la Universidad de Michigan². Estas investigaciones abordan distintos enfoques sobre los sistemas de recomendación de ropa, utilizando técnicas de IA, visión por computadora y aprendizaje profundo.

Una de las investigaciones de 2018 titulada “*Personalized Outfit Recommendation with Deep Neural Network*”[12] que propone un sistema de recomendación de outfits completos, es decir, conjuntos que combinan múltiples prendas como camisas, pantalones, polleras y zapatos.

El sistema llamado FashionNet usa una red neuronal convolucional profunda para extraer características visuales de las imágenes de las prendas; esto ayudaría a implementar lo mismo para definir el tipo de prenda que se encuentra en la imagen (tipo, color, etc.).

También realiza un análisis de compatibilidad, empleando una red neuronal para calcular la compatibilidad entre las diferentes prendas, evaluando qué tan bien combinan entre sí. Este modelo fue entrenado usando un conjunto de datos de la plataforma Polyvore³, que contenía conjuntos de ropa creados por usuarios, reflejando estilos y combinaciones populares de esa comunidad en línea.

Dentro de las consideraciones culturales, Polyvore tiene tendencias de moda de personas de Occidente, y por tal motivo no podría tomarse como estándar de moda para cualquier región, ya que usan prendas y combinaciones diferentes a las utilizadas en Uruguay, principalmente.

Otra de las tesis relevantes para este estudio es “*Creating Capsule Wardrobes from Fashion Images*” [4] se enfoca en la creación de *capsule wardrobes* (armarios cápsula), que es un conjunto de ropa mínima que permite múltiples combinaciones entre sí.

Basando su trabajo en un entrenamiento no supervisado utilizando imágenes de cuerpo completo de entornos reales. También aborda la eficiencia de subconjuntos, lo que modela la creación de armarios cápsula como un problema de selección de subconjuntos, utilizando funciones que capturan la compatibilidad, versatilidad y preferencias del usuario. Se emplea un algoritmo iterativo para maximizar las funciones de manera eficiente.

Un trabajo de 2019 titulado “*Fashion Outfit for E-Commerce*” [10] se acerca más a los objetivos de la startup que brinda los datos para este trabajo, que es la generación de un OUTFIT en base a un “*producto semilla*” (seed product) recomendando automáticamente otros productos compatibles para formar un conjunto coherente y estilísticamente atractivo.

Para este trabajo, se utilizó un modelo de red neuronal multicapa que integra características visuales y textuales de los productos para aprender de los estilos. Los autores del paper lo llamaron “*GORDN*”. En base a ello, los productos compatibles se agrupan cercanamente, facilitando la generación de conjuntos. El entrenamiento de esta IA fue supervisado, en la cual se le dieron datos predefinidos como “*Negative*” y “*Positive*”, siendo malas combinaciones y buenas combinaciones,

¹<https://www.cornell.edu>

²<https://espanol.umich.edu/>

³<https://polyvore.ch>

respectivamente.

Este modelo fue entrenado utilizando el *ASOS outfits dataset*⁴, un conjunto de datos que contiene una gran cantidad de conjuntos creados por estilistas profesionales, y que los autores han puesto a disposición de la comunidad de investigadores.

Otra investigación publicada en 2020 llamada "*Fashion Recommendation and Compatibility Prediction Using Relational Network*" [11] plantea un objetivo de no solo comparar pares de prendas, sino que se basa en un conjunto completo de ropa usando redes relacionales (Relational Networks). En este contexto, diseñan "*FashionRN*", un modelo que considera un outfit como una escena visual compuesta por múltiples objetos (las prendas). En lugar de extraer características globales como en trabajos anteriores, utilizan una arquitectura *DenseNet* (red neuronal convolucional densamente conectada) para convertir cada prenda en un vector de características; emplean una red relacional para analizar las relaciones no lineales entre los pares posibles dentro del conjunto. *Fashion RN* se compone de dos etapas principales:

- **Construcción de relaciones**, donde se aprenden las interacciones entre pares de prendas.
- **Evaluación de compatibilidad**, en la que se combinan todas las relaciones para obtener una puntuación que refleja la armonía del outfit completo.

Un artículo publicado en la página web de la Universidad de Michigan⁵ relacionado a la tienda de ropa Zara [9] hace mención a la importancia de la IA para predecir tendencias, gestionar los inventarios y la logística. La experiencia al cliente y la publicidad personalizada marcan la relevancia que tienen estas nuevas herramientas a la hora de mejorar el rendimiento de las organizaciones.

⁴ASOS Dataset en Kaggle

⁵<https://espanol.umich.edu/>

Capítulo 3

Marco Teórico

En este capítulo abordaremos algunos conceptos necesarios para interpretar correctamente las actividades que se realizarán en el la limpieza y entrenamientos necesarios para cumplir con los objetivos propuestos.

3.1 Conceptos teóricos

■ Categorías en moda y estandarización

- Problemas asociados a la falta de estándares en catálogos de productos.
- Importancia de la estandarización para análisis a gran escala o recomendaciones.
- Estrategias para clasificar y agrupar productos de distintas tiendas bajo una misma taxonomía.

■ Aprendizaje automático (Machine Learning)

- Concepto general de ML con Modelos Supervisados y No Supervisados.
- Aplicación en clasificación de productos, agrupamiento de tiendas o análisis de comportamiento.

■ Procesamiento de lenguaje natural (NLP)

- Clasificación de textos (descripciones de productos) usando modelos de lenguaje como facebook/bart-large-mnli[3] o modelos de Microsoft Azure OpenAI [6].
- Fundamentos del modelado semántico y embeddings de texto.
- Ejemplos de uso en clasificación de categorías a partir de textos no estructurados (nombre, descripción, categoría).

■ Visión por Computadora (Computer Vision)

- Definición y aplicaciones en moda: detección de prendas, extracción de características visuales.
- Uso de modelos preentrenados para tareas como clasificación de imágenes (ej: CNNs, ResNet, etc.).
- Uso de embeddings visuales para agrupar productos similares.

3.2 Cálculo de Similitud con vectores en pgvector

Para llegar a un entendimiento de lo que viene más adelante de cómo comparamos las diferentes prendas con sus embeddings, es necesario explicar el método utilizado que es la distancia coseno. Este método ayuda a identificar qué tan lejos se encuentran dos vectores, pero en relación

a su ángulo (*ver figura 3.1*). En base a eso, es que se determina la similitud entre dos vectores en comparación con el resto. Cuanto más pequeño sea ese ángulo, más similares son.

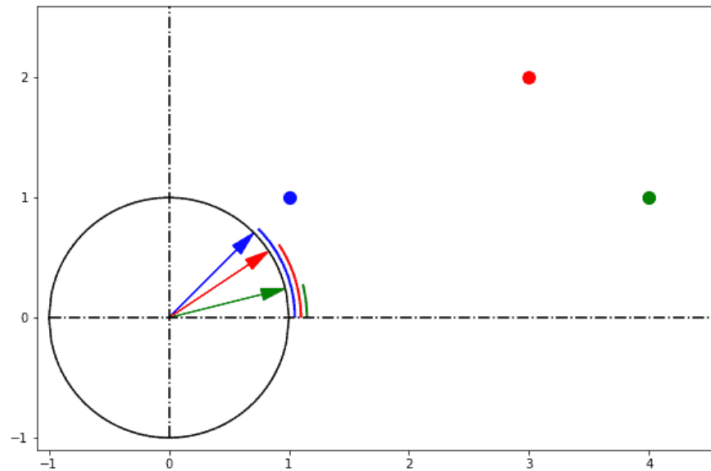


Figura 3.1: Gráfico bidimensional que representa los ángulos formados entre los versores correspondientes a los puntos, en función de la distancia coseno.

El sistema utiliza la extensión pgvector de PostgreSQL para calcular similitudes entre vectores de embeddings. La implementación emplea el operador de distancia coseno estándar para obtener medidas de similitud normalizadas.

3.2.1 Operadores de Distancia Coseno en pgvector

PostgreSQL con pgvector proporciona dos operadores principales para distancia coseno:

- `<#>`: Retorna la distancia coseno negativa en el rango $[-1, 1]$
- `<=>`: Retorna la distancia coseno estándar en el rango $[0, 2]$

3.2.2 Conversión de Distancia a Similitud

Para convertir la distancia coseno a una medida de similitud normalizada en el rango $[0, 1]$, se aplica la siguiente transformación:

$$\text{similitud} = 1 - \frac{\text{distancia_coseno}}{2} \quad (3.1)$$

Donde la distancia coseno se obtiene mediante el operador `<=>`.

3.2.3 Interpretación de Resultados

La función de transformación mapea los valores de distancia a similitud de la siguiente manera:

$$\text{distancia} = 0 \Rightarrow \text{similitud} = 1 - \frac{0}{2} = 1,0 \quad (\text{vectores idénticos}) \quad (3.2)$$

$$\text{distancia} = 2 \Rightarrow \text{similitud} = 1 - \frac{2}{2} = 0,0 \quad (\text{vectores ortogonales}) \quad (3.3)$$

3.2.4 Implementación SQL con similitud coseno (`<=>`) en pgvector

La consulta SQL implementa esta transformación para embeddings de texto e imagen:

```

SELECT
  (1 - (p.img_embedding <=> r.text_embedding_ref) / 2) AS
  similitud_imagen,
  (1 - (p.embedding <=> r.text_embedding_ref) / 2) AS similitud_texto
FROM product_embeddings p
JOIN referencia r ON true
ORDER BY p.embedding <=> r.text_embedding_ref;

```

3.2.5 Similitud Combinada

Para obtener una medida unificada de similitud, el sistema calcula la similitud combinada como:

$$\text{similitud_combinada} = \text{máx}(\text{similitud_texto}, \text{similitud_imagen}) \quad (3.4)$$

Esta aproximación prioriza la mejor coincidencia disponible entre las modalidades de texto e imagen, proporcionando un indicador robusto de relevancia del producto.

3.2.6 Rangos de Interpretación

Los valores de similitud se interpretan según los siguientes rangos:

Rango	Interpretación
[0,9, 1,0]	Muy similar (excelente coincidencia)
[0,7, 0,9)	Similar (buena coincidencia)
[0,5, 0,7)	Moderadamente similar
[0,3, 0,5)	Poca similitud
[0,0, 0,3)	Muy diferente

Cuadro 3.1: Interpretación de valores de similitud

3.3 Operadores de Distancia Euclidiana (L2) en pgvector

La **distancia euclidiana** es una de las métricas más utilizadas para medir la separación entre dos puntos en un espacio vectorial. Representa la *longitud del segmento de recta* que une dichos puntos, y corresponde a la distancia habitual que percibimos en el espacio geométrico.

Dada dos observaciones o vectores:

$$\vec{A} = (a_1, a_2, \dots, a_n) \quad \text{y} \quad \vec{B} = (b_1, b_2, \dots, b_n)$$

la distancia euclidiana entre ambos se define como:

$$d(\vec{A}, \vec{B}) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$$

Esta métrica corresponde a la **norma L_2** del vector diferencial, es decir:

$$d(\vec{A}, \vec{B}) = \|\vec{A} - \vec{B}\|_2$$

Interpretación geométrica:

- Mide la distancia “en línea recta” entre los puntos $\vec{A}$ y $\vec{B}$.

- Cuanto menor sea el valor, más cercanos estarán los vectores en el espacio.
- Es sensible a la escala de las variables, por lo que suele requerir normalización de los datos antes de su uso.

Ejemplo en dos dimensiones:

$$d(\vec{A}, \vec{B}) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

En este caso, la distancia representa la hipotenusa del triángulo formado por las diferencias entre las coordenadas en los ejes x e y , según el teorema de Pitágoras (*ver Figura 3.2*).

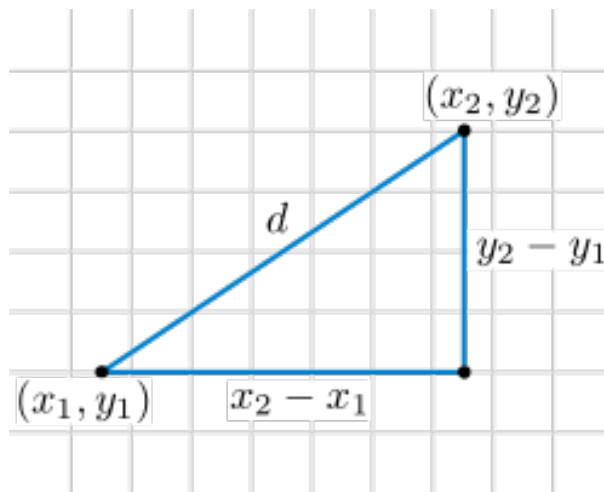


Figura 3.2: Ejemplo de la distancia euclidiana en dos dimensiones.

3.3.1 Implementación SQL de la distancia Euclidiana

Para el caso de las pruebas para medir la distancia euclidiana en nuestro dataset en PostgreSQL con la librería de pgvector debemos realizarlo de esta forma:

```

1      WITH referencia AS (
2          SELECT %s::vector AS embedding_ref
3      )
4      SELECT
5          p.product_id,
6          p.text_for_embedding::json ->> 'categoría' AS categoria,
7          p.text_for_embedding,
8          p.img_base64,
9          1 - (p.embedding <-> r.embedding_ref) AS similitud
10     FROM public.product_embeddings p
11     JOIN referencia r ON true
12     ORDER BY p.embedding <-> r.embedding_ref
13     LIMIT %s;

```

3.4 Optimización de Búsqueda Vectorial: Índices HNSW

PostgreSQL con la extensión **pgvector** [13] soporta búsqueda vectorial mediante el operador de distancia coseno ($\langle=\rangle$). El sistema actualmente utiliza **búsqueda exhaustiva** $O(n)$ sin índices HNSW debido al tamaño del catálogo (764 productos), donde la latencia promedio de $<200\text{ms}$ cumple requisitos de experiencia de usuario.

Para catálogos mayores a 5,000 productos, pgvector implementa índices **HNSW (Hierarchical Navigable Small World)** [5] que reducen la complejidad de búsqueda de $O(n)$ a $O(\log n)$, con latencias estimadas de 20-50ms.

Arquitectura HNSW HNSW organiza vectores en un grafo navegable multinivel inspirado en redes de “mundo pequeño” (*small-world networks*), donde cualquier par de nodos es alcanzable mediante pocos saltos:

- **Capas superiores:** Conexiones globales dispersas que permiten saltos largos en el espacio vectorial para búsqueda inicial rápida (greedy search).
- **Capas inferiores:** Conexiones locales densas que refinan iterativamente hasta localizar los k vecinos más cercanos con mayor precisión.

El algoritmo de búsqueda comienza en la capa superior y desciende progresivamente, afinando la navegación en cada nivel hasta recuperar los k vecinos más similares con complejidad promedio $O(\log n)$, frente a $O(n)$ de búsqueda exhaustiva.

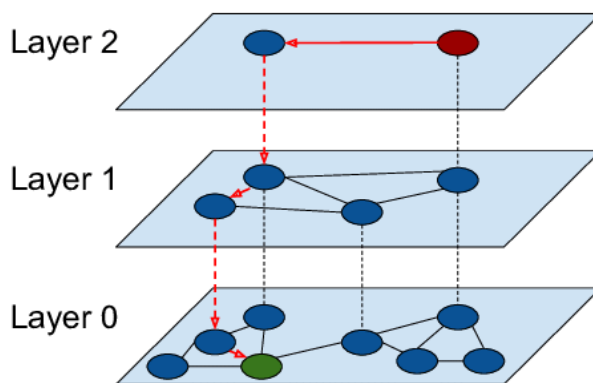


Figura 3.3: Arquitectura jerárquica del índice HNSW. La búsqueda comienza en Layer 2 (conexiones globales), desciende a Layer 1 (regionales) y finaliza en Layer 0 (locales densas) para recuperar vecinos más cercanos. Este diseño equilibra eficiencia ($O(\log n)$) con precisión en espacios de alta dimensión. Adaptado de [5].

Justificación de No Implementación La decisión de no implementar índices HNSW en el sistema actual se fundamenta en los siguientes criterios técnicos:

1. **Tamaño del catálogo:** Con 764 productos, la búsqueda exhaustiva completa cada consulta en $<200\text{ms}$, cumpliendo el límite de experiencia de usuario (300ms). La ganancia de 150ms que ofrecería HNSW (latencia esperada: 20-50ms) no justifica la complejidad adicional.
2. **Complejidad operacional:** Índices HNSW requieren:
 - Mantenimiento durante inserciones/actualizaciones de productos
 - Reconstrucción periódica para mantener calidad del grafo
 - Configuración de 2 hiperparámetros (m , $ef_construction$) que afectan trade-off memoria-precisión
 - Overhead de memoria del 20-30 % por índice (con 3 embeddings por producto: imagen, texto ES, texto EN $\rightarrow$ 3 índices necesarios)
3. **Precisión vs velocidad:** HNSW es un método de búsqueda *aproximada* (ANN - Approximate Nearest Neighbors):
 - Recall típico: 95-98 % (pérdida de 2-5 % de resultados óptimos)
 - Búsqueda exhaustiva: 100 % recall (resultados exactos matemáticamente garantizados)

En un catálogo pequeño donde la búsqueda exacta es computacionalmente viable, sacrificar precisión por velocidad marginal no es estratégicamente óptimo.

4. **Escalabilidad planificada:** El diseño modular del sistema permite migración futura a HNSW mediante ejecución de script SQL sin modificaciones en la lógica de negocio, preservando flexibilidad arquitectónica.

Umbral de implementación recomendado:

Cuadro 3.2: Comparación: Búsqueda exhaustiva vs índices HNSW

Aspecto	Búsqueda Exhaustiva (actual)	Índices HNSW (propuesta)
Complejidad temporal	$O(n)$	$O(\log n)$
Latencia (764 productos)	<200ms	20-50ms
Latencia (10,000 productos)	$\approx 2,600\text{ms}$	50-80ms
Overhead de memoria	0 %	20-30 % por índice
Mantenimiento	Ninguno	Reconstrucción periódica
Configuración requerida	Ninguna	2 hiperparámetros
Índices necesarios	0	3 (uno por embedding)
Recomendado para	< 5,000 productos	> 5,000 productos

Conclusión técnica:

La decisión de usar búsqueda exhaustiva se fundamenta en el principio de ingeniería de software de evitar optimización prematura. Con 764 productos, la búsqueda lineal cumple requisitos de latencia (<200ms) y garantiza precisión máxima (100 % recall), mientras que HNSW introduciría complejidad adicional (mantenimiento de 3 índices, configuración de hiperparámetros, pérdida de 2-5 % de recall) sin beneficio proporcional en el contexto actual. El sistema fue diseñado modularmente para permitir migración futura a HNSW cuando el crecimiento del catálogo justifique la inversión técnica (umbral recomendado: 5,000+ productos o requisitos de latencia <100ms).

3.5 Embeddings

En este proyecto se implementó el almacenamiento de información en vectores (embeddings) para optimizar las búsquedas y estandarizar datos no estructurados procedentes de las tiendas y de los usuarios.

Los embeddings son representaciones numéricas (vectores) de objetos como palabras, oraciones, descripciones, imágenes o señales auditivas. Su objetivo es capturar relaciones semánticas y contextuales: vectores cercanos en el espacio representan elementos con significado o características similares (por ejemplo, “auto” y “camioneta” estarán más próximos entre sí que con “conejo”).

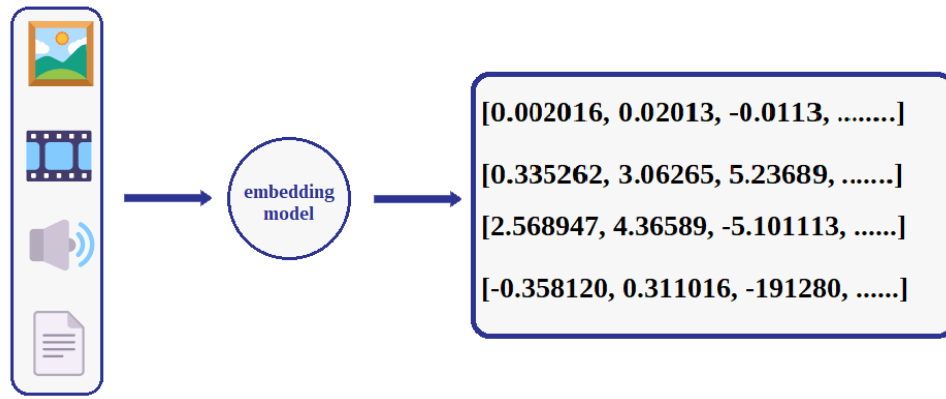


Figura 3.4: Diagrama de flujo de transformación de datos no estructurados a embeddings.

Características relevantes:

- Dimensionalidad: los embeddings tienen dimensión fija (por ejemplo 512), y mayor dimensión puede permitir una representación más matizada.
- Consistencia de modelo: es crítico comparar embeddings generados con el mismo modelo y configuración; distintos modelos (o tokenizaciones) producen espacios vectoriales incompatibles.
- Multimodalidad: existen embeddings de texto e imagen; cuando se usan ambos, conviene definir una estrategia de combinación (concatenación, ponderación, re-ranking).

En nuestro trabajo, los embeddings almacenan las descripciones de las prendas y las imágenes (estas últimas convertidas a Base64). Las consultas de los usuarios se transforman al mismo espacio vectorial (misma arquitectura de embedding) para calcular similitudes y recuperar los productos más relevantes.

3.6 RAG (Retrieval Augmented Generation)

Cuando se emplean modelos grandes (LLMs) resulta necesario proporcionarles contexto actualizado y específico del dominio. Retrieval Augmented Generation (RAG) es una arquitectura que combina un componente de recuperación de información con un generador (LLM) para mejorar la precisión y reducir las “alucinaciones”.

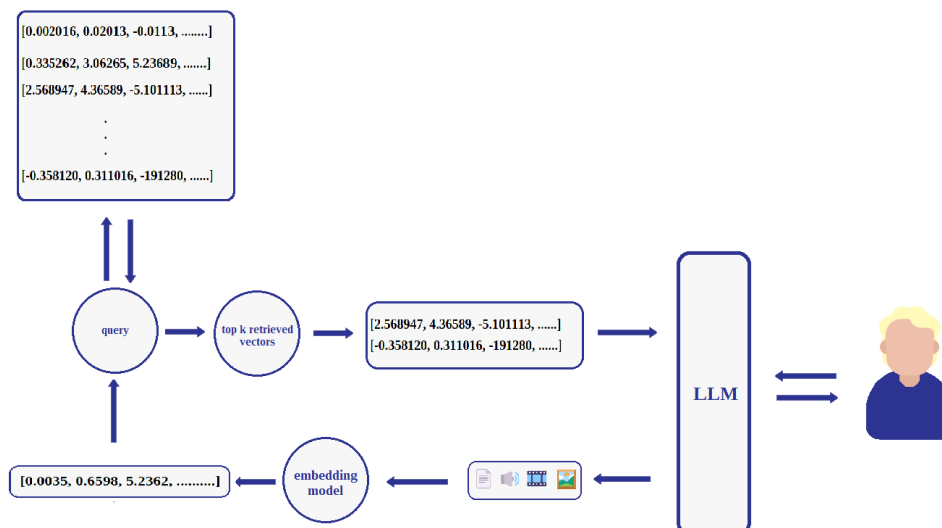


Figura 3.5: Arquitectura RAG vinculada con un LLM en una interfaz de usuario.

Flujo típico en RAG:

1. El usuario formula una consulta.
2. Se genera un embedding de la consulta y se recuperan los documentos/fragmentos más similares desde el índice vectorial (top-k).
3. Opcionalmente se aplica un re-ranking (por ejemplo con un LLM o métricas adicionales).
4. El LLM recibe la consulta junto con los fragmentos recuperados como contexto y genera la respuesta condicionada en esa información.

Ventajas:

- Mejora la precisión al basar respuestas en datos verificables.
- Permite respuestas con información actualizada sin reentrenar el LLM.
- Reduce las alucinaciones al limitar la generación al contexto recuperado.

En este proyecto, RAG conecta el LLM con la base de datos de productos y sus embeddings para generar respuestas y recomendaciones fundamentadas en el catálogo real.

Capítulo 4

Metodología

4.1 Diseño del estudio

Este proyecto adopta un **enfoque experimental mixto** que combina investigación aplicada en ciencia de datos con desarrollo de sistemas de inteligencia artificial. el diseño metodológico se estructura en cuatro fases secuenciales.

4.1.1 Fases del Diseño Metodológico

El proyecto se divide en cuatro fases principales:

Fase 1: Limpieza y Preparación de Datos

1. Limpieza y Normalización

- Problema identificado: variabilidad en nombrar las categorías y los géneros en las tiendas
- Solución: Clasificación automática con Azure OpenAI GPT-4o-mini
- Validación: Revisión manual de 500 productos (muestra aleatoria)

2. Descarga de Imágenes

- Filtrado: Solo productos con URLs de imagen válidas
- Almacenamiento: Sistemas de archivos locales en formato Base 64
- Cobertura final: 764 productos (4.5 %) con imágenes disponibles

Fase 2: Generación de Embeddings Multimodales

1. Embeddings de Texto Bilingües

- Modelo: OpenFashion CLIP (especializado en moda)
- Estrategia: Concatenación de embeddings ES (512D) + EN (512D) = 1024D
- Justificación: Modelo pre-entrenado en inglés, necesidad de capturar términos locales
- Procesamiento: Traducción automática con Google Translator
- Infraestructura: API REST externa

2. Embeddings de Imagen

- Modelo: OpenFashion CLIP visual encoder (512D)
- Limitación identificada: Ruido contextual (fondos, múltiples prendas, modelos)
- Decisión de diseño: Excluir de entrenamiento CatBoost, usar solo para búsqueda visual

3. Almacenamiento Vectorial

- Base de datos: PostgreSQL 15 con extensión pgvector
- Almacenamiento: Se almacenan los embeddings de las descripciones en español, inglés y las imágenes en la tabla `product_embedding`

Fase 3: Entrenamiento de Modelos de Clasificación

1. Entrenamiento CatBoost

- Algoritmo: Gradient Boosting optimizado para datos categóricos
- Configuración:
 - Iterations: 1000 (early stopping en 50 rounds)
 - Depth: 6
 - Learning rate: 0.1
 - Loss function: MultiClass
 - Class weights: Balanced (automático con sklearn)
- Validación: Estratificada 5-fold cross-validation
- División: 80 % entrenamiento, 20 % prueba (stratified split)

2. Modelos Entrenados

- **Modelo 1 - Género:** 6 clases (Dama, Caballero, Niño, Niña, Bebé, Unisex)
- **Modelo 2 - Categoría:** 17 clases (Vestidos, Remeras, Pantalones, Buzos, etc.)
- Features: Vector de 1024D (embeddings concatenados ES+EN)

Fase 4: Desarrollo del Sistema Multiagente

1. Diseño de Arquitectura

- Patrón: Arquitectura con orquestación centralizada
- Agentes: 7 componentes especializados con responsabilidades únicas
- Comunicación: Event-driven con Streamlit session state como bus de mensajes

2. Implementación de Agentes

- CoordinadorAgent: Decisión de workflow con GPT-4o-mini (temp=0.1)
- AnalizadorConsultaAgent: Extracción de atributos con GPT-4o-mini (temp=0.2)
- ImageAnalyzerAgent: Descripción de prendas con GPT-4 Vision (temp=0.3)
- ClasificadorCatBoostAgent: Predicción con modelos ML locales
- BuscadorAgent: Queries SQL + pgvector con búsqueda híbrida
- EspecialistaOutfitsAgent: Generación de combinaciones con GPT-4o-mini (temp=0.7)
- PresentadorResultadosAgent: Síntesis de respuesta con GPT-4o-mini (temp=0.3)

3. Optimizaciones Implementadas

- Memoria conversacional: Historial de sesión con concatenación de contexto
- Búsqueda híbrida: Búsqueda contra embeddings de texto y de imágenes con sistema de votación
- Prompts en BD: Tabla `ui_agent_prompts` para edición sin redeploy
- Auditoría completa: Logging de tokens, costos y latencias

4.2 Recopilación de datos

La información proporcionada por la empresa cuenta con cuatro archivos CSV (Coma Separated Values), obtenidos por medio de técnicas de *scraping* de la web de 35 tiendas del país, obteniendo diversos productos. El total de registros asciende a 50,000.

4.3 Preprocesamiento de datos

Los datos deben ser limpiados y organizados en función de las categorías presentes en los archivos originales. Estos datos resultan fundamentales para la investigación, dado que constituyen la base de los sistemas de búsqueda que se planifican implementar. Durante el proceso de iteración sobre la información, se resolvieron problemas comunes tales como la presencia de comas adicionales, procurando mantener la integridad de los datos, especialmente en aquellos casos donde una parte del contenido se encuentra codificada en formato JSON, utilizado para representar metadatos estructurados.

4.3.1 Análisis de categorías y géneros en las tiendas

Los artículos fueron identificados mediante un identificador global único denominado *product_id*, generado durante el proceso de scraping, el cual permite vincular de forma consistente la información distribuida en los cuatro archivos CSV originales.

Se identificaron las tiendas que efectivamente poseen artículos registrados en el conjunto de datos (ver figura 4.1), observando que únicamente 19 de las 35 tiendas iniciales contienen productos con información completa tras el proceso de filtrado.

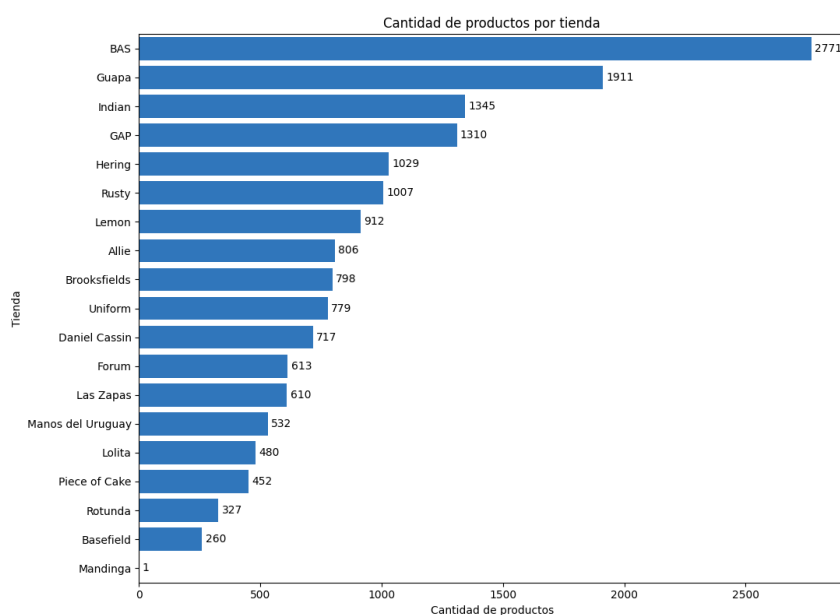


Figura 4.1: Tiendas con artículos registrados en el conjunto de datos

En relación con el sexo de las prendas, se observa que hay varias formas de definirlas en función de la tienda (ver figura 4.2), lo que presentaría dificultades a la hora de implementarlo dentro de un modelo si no se estandariza.

Otro detalle es que hay prendas en el DF que no cuentan con este dato, ya que algunas tiendas están dirigidas a damas, caballeros o niños, lo que hace que se deba utilizar técnicas de ML (Machine Learning) para identificar si la prenda corresponde a masculina, femenina o unisex. También se detectan prendas que son unisex, pero que se identifican en el scrapeo como una lista, ejemplo: Hombre, mujer.

Esto que se detalló, más los 5990 valores "Sin dato" hacen que se planifique la estandarización de género en esta columna.

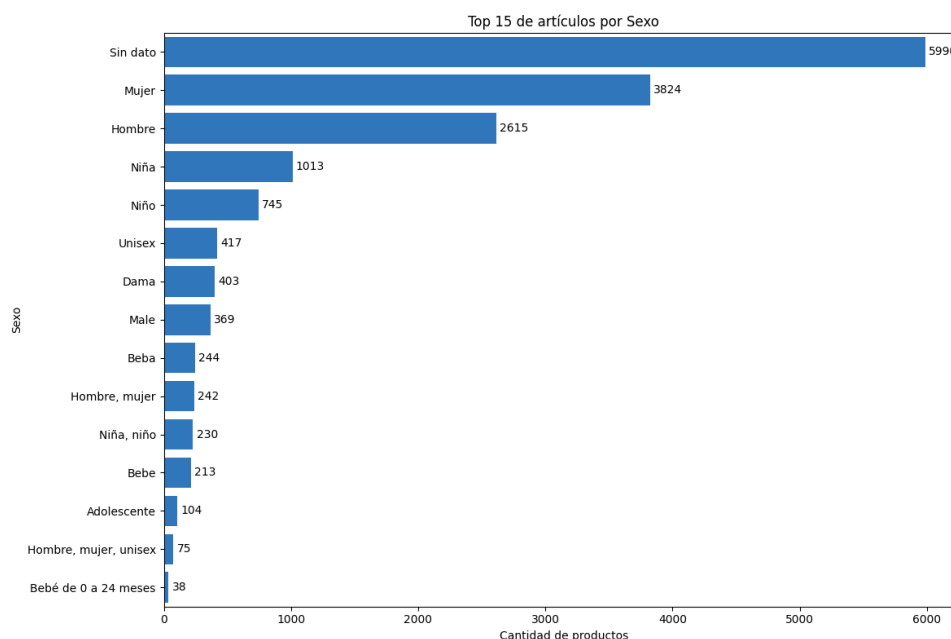


Figura 4.2: Top 15 productos por sexo

En la nube de palabras se puede apreciar que términos son los más utilizados por las tiendas del país para referirse al género de una persona (ver figura 4.3).



Figura 4.3: Distribución de los términos utilizados para definir el género del producto

También se encuentran artículos de las tiendas que no forman parte de indumentarias, cómo por ejemplo: marcadores, skates, tablas, golosinas (ver figura 4.4).

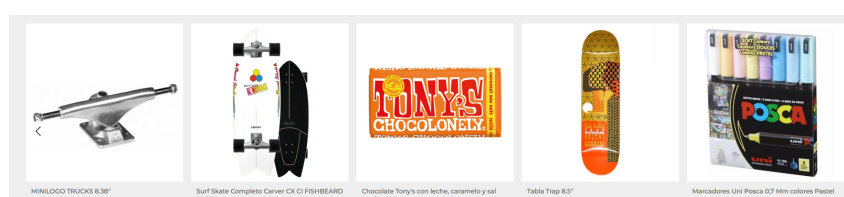


Figura 4.4: Productos que no forman parte de prendas, calzados o accesorios.

Fuente: Capturas obtenidas desde <https://laisla.com.uy/>.

En lo que respecta al catálogo, las tiendas manejan sus propias categorías y en relación a sus prendas (*ver figura 4.5*), nombran a conveniencia; por lo tanto, se debe ajustar en una “*Categoría Global*”. Para esto, se establecerá la información de la prenda y se iterará sobre las categorías globales establecidas.

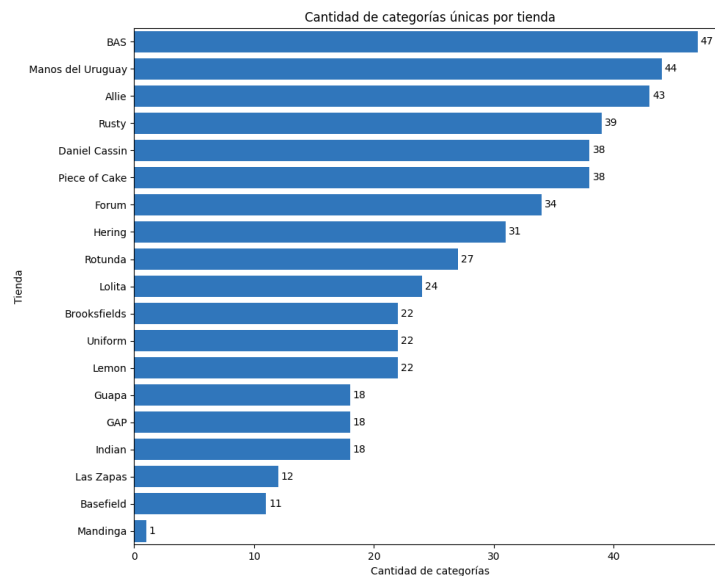


Figura 4.5: Cantidad de categorías que define cada una de las tiendas.

Si volvemos a realizar la nube de palabras para el caso de las categorías, nos encontraremos con la misma problemática que se nos presentó para el género (*ver figura 4.6*), diferentes tiendas utilizan diferentes terminologías para definir una misma categoría.



Figura 4.6: Variabilidad en las denominaciones de categorías en los catálogos analizados.

4.3.2 definición de colores por las tiendas

Otro de los inconvenientes detectados en los catálogos de las tiendas, es la forma en que define el color de las prendas. En muchos casos, las etiquetas de color no siguen un patrón consistente y se utilizan para fines diversos: codificar tipos de estampado, describir tonalidades de manera no estandarizada, incluir fragmentos de URL que contienen imágenes del tejido o, incluso, registrar códigos RGB. Esta heterogeneidad dificulta cualquier intento de normalización, por lo que resulta necesario recurrir a modelos LLM con capacidad de visión para identificar el color predominante de cada prenda (*ver figura 4.7*).

En el contexto de este proyecto, el color no será estandarizado, ya que se priorizará la normalización de las categorías y del género de las prendas para el posterior entrenamiento de modelos de machine learning.



Figura 4.7: Nube de palabras de categorías de colores reportados por las tiendas de ropa.

El inconveniente se ve mejor si invertimos las relevancias donde las palabras más grandes representan las que tienen menos presencia en el set de datos (ver figura 4.8).



Figura 4.8: Nube de palabras de categorías de colores reportados por las tiendas de ropa.

4.4 Proceso de limpieza y estandarización de datos

El proceso de limpieza y preparación de datos se estructuró en 6 etapas secuenciales documentadas en el *ANEXO I: LIMPIEZA DE DATOS.ipynb*. A continuación se detalla cada fase del pipeline de procesamiento.

4.4.1 Recopilación y Consolidación de Datos

Carga de Archivos Fuente

Se trabajó con cuatro archivos CSV base:

- **categories.csv**: Categorías de productos por tienda
- **product_data.csv**: Metadatos detallados (requirió parsing manual por estructura JSON embebida)

- **products.csv**: Información básica de productos
- **shops.csv**: Datos de tiendas online

Normalización y Unión Relacional

Previo a las uniones, se normalizaron las URLs eliminando prefijos `www.` y estandarizando protocolos. Se realizaron tres operaciones de merge secuenciales:

1. `shops + categories` → Asociación tienda-categoría
2. Resultado anterior + `products` → Incorporación de productos
3. Resultado anterior + `product_data` → Dataset completo

Dataset consolidado: 16,660 productos de 31 tiendas online.

4.4.2 Clasificación Automática con LLM

Recuperación de Imágenes

Se utilizaron 764 imágenes previamente descargadas almacenadas en `media/products/`, con conversión a Base64 y redimensionamiento a 224×224 píxeles para procesamiento uniforme.

4.4.3 Estandarización de Categorías Globales

Se determina que es necesario generar un listado de categorías globales que contemplen todos aquellos artículos que son similares. La primera técnica aplicada es la de unificar todo en una misma columna lo que es nombre, categoría y descripción para iterar este nuevo campo llamado `input_text` en el DF. Esta nueva columna será la que se comparará con el modelo *facebook/bart-large-mnli*[3] y con *Azure OpenAI*[6].

El listado de categorizaciones globales que se plantea que se definió consta de varios artículos que se encuentran dentro del área de estudio.

Categorías globales utilizadas:

- | | | |
|--------------|------------------|------------------|
| ▪ Camisas | ▪ Enteritos | ▪ Calzado |
| ▪ Blusas | ▪ Jumpsuits | ▪ Zapatillas |
| ▪ Remeras | ▪ Rompers | ▪ Sandalias |
| ▪ Tops | ▪ Ropa interior | ▪ Botas |
| ▪ Sweaters | ▪ Pijamas | ▪ Accesorios |
| ▪ Buzos | ▪ Medias | ▪ Carteras |
| ▪ Chalecos | ▪ Bikinis | ▪ Necessaire |
| ▪ Pantalones | ▪ Trajes de baño | ▪ Conjuntos |
| ▪ Shorts | ▪ Ropa deportiva | ▪ Joyas |
| ▪ Faldas | ▪ Abrigos | ▪ No clasificado |
| ▪ Vestidos | ▪ Camperas | |
| ▪ Monos | ▪ Chaquetas | |

Categorías globales de género:

- Dama
- Niño
- Bebés
- Caballero
- Unisex
- No encontrado

4.4.4 Conversión de imágenes a Base64 para almacenamiento y procesamiento

Con el objetivo de incorporar la información visual de los productos al modelo de inteligencia artificial, se implementó un proceso de conversión de imágenes al formato Base64. Esta codificación permite representar las imágenes como cadenas de texto, lo que facilita su almacenamiento en bases de datos y su transmisión en flujos de datos estructurados, como JSON o CSV.

Para optimizar el procesamiento y reducir el volumen de datos, se seleccionó únicamente la primera imagen disponible por cada artículo. Esta imagen se considera representativa del producto y se utiliza como entrada visual principal en el modelo. La conversión se realiza directamente desde la URL mediante una solicitud HTTP, codificando el contenido binario de la imagen en formato Base64.

Al integrar las imágenes codificadas junto con los metadatos de cada prenda (como descripción, categoría, color y género), se enriquece la entrada del modelo, lo cual puede mejorar significativamente la precisión en la clasificación y recomendación de productos. Este enfoque también resulta fundamental para la implementación futura de motores de búsqueda por similitud y sistemas de recomendación basados en embeddings visuales.

Al realizar la conversión de las imágenes a Base64, se optó por reducir el tamaño de la imagen para optimizar la conversión, evitando que al reducir se pierda información de la imagen cuando se generen los embeddings (*ver figura 4.9*).

```
1 def imagen_a_base64_redimensionada(ruta, max_size=(224, 224)):  
2     """  
3     Convierte una imagen a base64 después de redimensionarla  
4     sin deformarla.  
5     """  
6     img = Image.open(ruta).convert("RGB")  
7     # Redimensionar manteniendo proporción  
8     img.thumbnail(max_size, Image.LANCZOS)  
9  
10    buffered = io.BytesIO()  
11    img.save(buffered, format="JPEG")  
12    img_base64 = base64.b64encode(buffered.getvalue()).decode()  
13    return f"data:image/jpeg;base64,{img_base64}"
```



Figura 4.9: Imagen redimensionada convertida a base64. Fuente: <https://www.allie.com.uy>

Al tener problemas para poder obtener la mayor cantidad de imágenes de las tiendas por protecciones anti-scraping de las mismas, se optó por usar un listado de 773 prendas previamente descargadas, evitando perder artículos por falta de imágenes.

4.4.5 Uso del modelo facebook/bart-large-mnli

El modelo utilizado para la clasificación fue *facebook/bart-large-mnli*, una variante del modelo BART desarrollado por Facebook AI. Este modelo se basa en la arquitectura Transformer y posee aproximadamente 400 millones de parámetros. Este en particular ha sido entrenado específicamente para realizar tareas de comprensión y generación de texto y para la clasificación textual [3].

En base a las categorías definidas en la sección anterior, se evaluó el desempeño del modelo en la clasificación de descripciones de productos. Los resultados obtenidos mostraron una baja precisión en la asignación de categorías, ya que el modelo no logró asociar correctamente la categoría global con la descripción correspondiente. Esto hace que su uso sea irrelevante para el propósito del estudio. A continuación, se muestra un resumen con ejemplos representativos de las categorías originales y las predicciones correctas de una muestra de 50 registros (*Ver cuadro 4.1*) y las incorrectas (*Ver cuadro 4.2*) realizadas por el modelo, que al parecer el modelo tiende a confundirse con ciertos términos o cuando el contexto no es explícito, la respuesta no es acertada.

Cuadro 4.1: Ejemplos de categorías originales y categorías globales predichas por el modelo

name_category	input_text	categoria_global_predicha
Abrigos	Abrigos	Abrigos
Chaquetas	Chaquetas	Chaquetas
Blusas	Blusas de seda	Blusas
Zapatillas deportivas	Zapatillas para correr	Calzado
Billeteras	Billeteras de cuero	Accesorios

Cuadro 4.2: Ejemplos de predicciones incorrectas del modelo

name_category	input_text	categoria_global_predicha	Observación
Babuchas	Babuchas	Trajes de baño	Incorrecta
Billeteras	Billeteras	Blusas	Incorrecta

4.4.6 Uso del modelo OpenAI en Azure

Para el caso de la misma muestra, usando OpenAI de Azure, obtiene una respuesta más acertada a la categoría global propuesta (*Ver cuadro 4.3*).

Cuadro 4.3: Ejemplos de clasificación de productos por el modelo de OpenAI (Azure)

input_text	categoría_clasificada
Calzado Bota pu - Marrón	Botas
Bikinis y mallas BOTTOM BASIC LIRIO Parte inferior	Bikinis
Calzado Pantufla - Camel	Calzado
Necessaire Pecarí Rojo	Accesorios
CAMISA DIAGONAL	Camisas
BLUSA PUMPKINS Blusa de manga larga	Blusas
ANILLO KARAL	Joyas
Pantubota - Negra	Calzado
Remera oversize - Mundo Gris Oscuro	Remeras
BLAZER MUREZ Blazer plano de manga larga	Chaquetas

4.4.7 Uso de Open AI para determinar las categorías y el género de las prendas

Dado los buenos resultados que obtenemos con el modelo de Open AI; se establece utilizar dicho modelo para definir las categorías de las prendas y también el género en una misma consulta, para optimizar el rendimiento del modelo. Se predefine un prompt y un listado de categorías y géneros preestablecidos para facilitar la organización de los datos. Se utilizó temperatura 0.0 (determinística) para clasificación binaria. El input consistió en la concatenación de:

```
1 name_category + product_name + product_description + product_sex
```

Resultados: 764 productos clasificados exitosamente. Los productos no clasificados o con errores fueron excluidos del dataset final.

```

1 PROMPT:
2 Actúa como un experto en moda y clasificación de productos de ropa.
3 Tu tarea es identificar el género y la categoría general de una prenda,
4 basándote únicamente en el texto proporcionado.
5 Solo puedes elegir entre las siguientes opciones:
6 - Géneros posibles: {'', ' '.join(etiquetas_genero)}.
7 - Categorías posibles: {'', ' '.join(etiquetas_globales)}.
8 Si no puedes determinar el género, responde con: No encontrado.
9 Si no puedes determinar la categoría, responde con: No clasificado.
10 Descripción del producto: {input_text}
11
12 Responde estrictamente en el siguiente formato (sin agregar
13 explicaciones ni texto adicional):
14 Género: [una opción exacta]
15 Categoría: [una opción exacta]
```

Como se muestra en la Figura 4.10, los productos clasificados por categoría y sexo presentan una distribución clara, obteniendo pocas categorías y sexos identificados con el prompt actual. Se estudiarán los datos categorizados, los errores y los no categorizados, ya que algunos deben ser eliminados por no ser parte de los outfits.

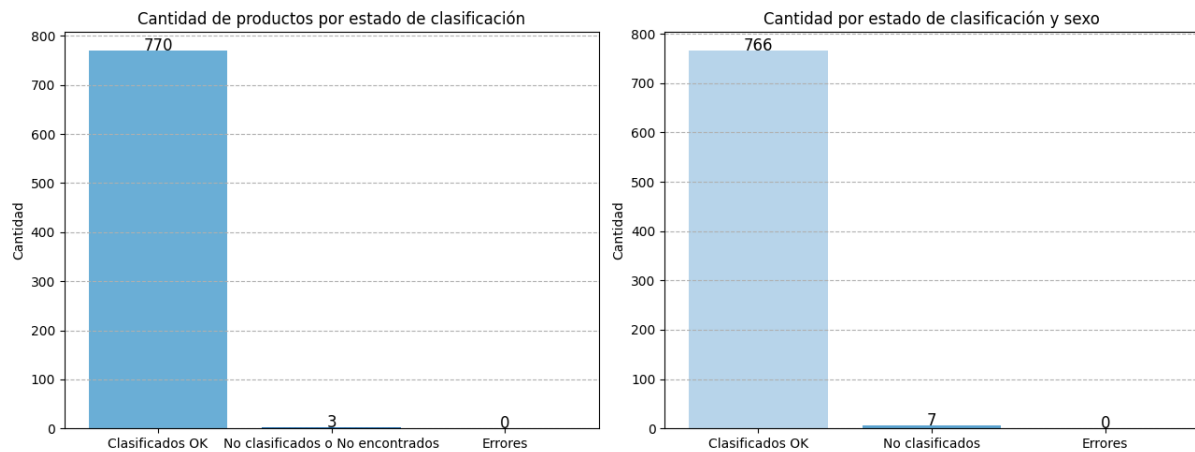


Figura 4.10: Resultado de categorías y géneros globales obtenidos.

De una base de 773 artículos se logró reetiquetar 756 solamente usando la columna generada por las descripciones de los productos, evitando utilizar las imágenes en esta etapa, ya que en algunos casos en las fotos aparecen prendas adicionales, generando un outfit con la prenda definida en el catálogo.

Dado que la proporción de artículos que no fue determinada su categoría o género fué muy baja, se opta por descartarlos simplemente y seguir con el estudio ya que no es representativa para el trabajo.

4.4.8 Preparación de base de datos para almacenar la información obtenida

Se prepara una base de datos en PostgreSQL alojada en docker, instalando la librería pgvector para manejo y almacenamiento de vectores en la base de datos. La estructura cuenta con los artículos escrapeados de las tiendas y los artículos convertidos (*ver figura 4.11*) y filtrados como elementos a utilizar por el modelo (*ver figura 4.12*).

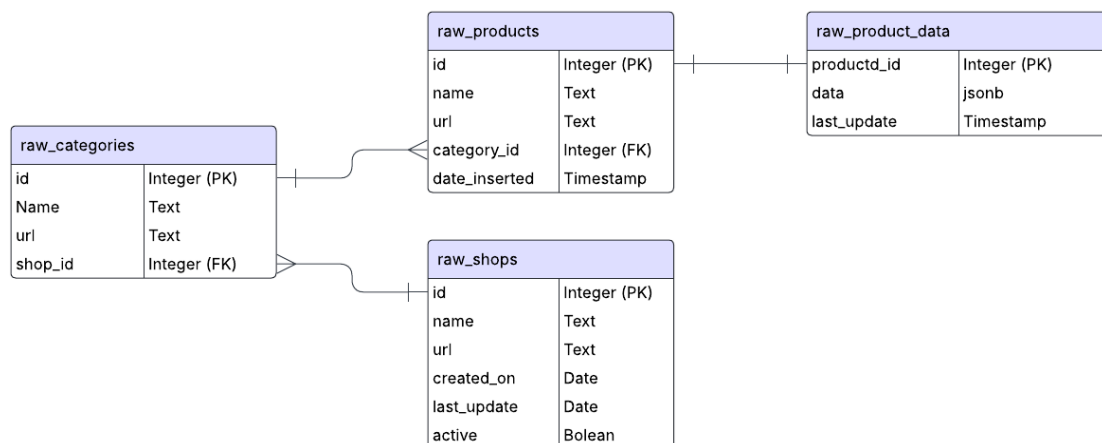


Figura 4.11: Tablas de los artículos obtenidos de las tiendas.

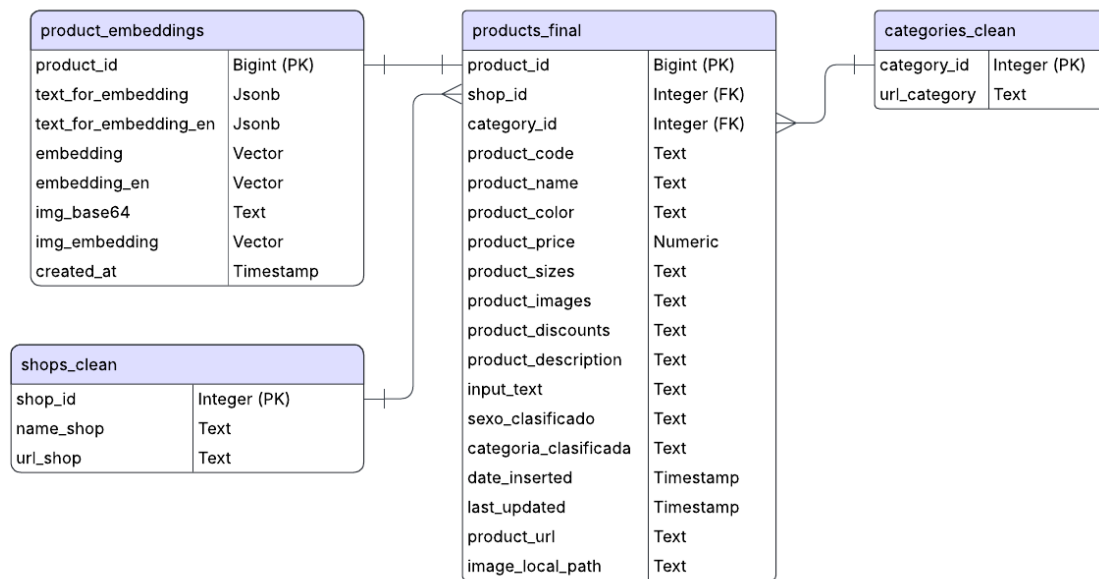


Figura 4.12: Tablas generadas con las re categorizaciones y los embeddings necesarios.

Estrategia de Inserción

4.4.9 Preparando los textos para generar los embeddings

Se crean las columnas `text_for_embedding` y `text_for_embedding_en` en la base de datos para poder almacenarlos. Para generar las traducciones de los textos que se encuentran en español en la columna `text_for_embedding` en la tabla `product_embeddings`, antes de generar los embeddings en inglés, se usa Google Translate por medio de la librería `deep_translator` importando `GoogleTranslator`. Logrado esto, lo almacenamos en la misma tabla bajo el nombre `text_for_embedding_en` con los mismos campos: Color, nombre, género, categoría y descripción.

4.5 Generación de embeddings de texto e imágenes

Se generaron embeddings tanto en español como en inglés para comparar su eficacia, dado que OpenFashionCLIP [2] fue entrenado principalmente en inglés. Para mantener la coherencia en los cálculos de similitud, todos los embeddings —tanto textuales como visuales— se regeneraron con el modelo OpenFashionCLIP, el cual produce representaciones de 512 dimensiones. Debido al idioma de entrenamiento, se espera que las consultas en inglés presenten una mayor precisión que las realizadas en español. Los embeddings visuales se obtuvieron procesando los bytes de las imágenes decodificadas desde Base64 mediante el mismo modelo.

Esquema de Embeddings

La tabla `product_embeddings` almacena:

```

1 CREATE TABLE product_embeddings (
2     embedding vector(512),          -- Texto español
3     embedding_en vector(512),       -- Texto inglés
4     img_embedding vector(512),      -- Imagen
5     text_for_embedding JSONB,       -- Metadata ES
6     text_for_embedding_en JSONB    -- Metadata EN
7 );

```

Listing 4.1: Campos vectoriales en PostgreSQL

4.6 Pruebas de búsquedas por medio de la distancia coseno en la base de datos

Se adoptó el operador de **distancia coseno** (`<=>`) como métrica principal:

- **Rango:** $[0, 2]$, donde valores menores indican mayor similitud
- **Interpretación:** 0-0.3 (muy similar), 0.3-0.7 (similar), >0.7 (poco similar)

Para realizar las búsquedas, se generará un índice para buscar por medio de la distancia coseno de los productos. y se ensayan tres situaciones, siempre tomando los 5 mejores.

Modalidades de Búsqueda

Se implementaron 5 estrategias de búsqueda vectorial:

1. Texto → Texto (ES):

```

1 SELECT * FROM product_embeddings
2 ORDER BY embedding <=> query_embedding LIMIT 20;

```

2. Texto → Texto (EN):

```

1 ORDER BY embedding_en <=> query_embedding_en

```

3. Imagen → Imagen:

```

1 ORDER BY img_embedding <=> query_img_embedding

```

4. Texto → Imagen (cross-modal):

```

1 ORDER BY img_embedding <=> text_embedding_query

```

5. Búsqueda híbrida con filtros:

```

1 WHERE text_for_embedding::json ->> 'categoría' = 'Remeras'
2     AND text_for_embedding::json ->> 'género' = 'Caballero'
3 ORDER BY embedding <=> query_embedding;

```

En el primer caso, el texto que ingresa la persona está en español y se buscan las prendas. Antes de la búsqueda, se genera el embedding de lo que la persona ingresa para buscar la distancia coseno. Primera prueba fue contra la columna embedding:

Texto ingresado: “Remera azul manga corta para hombre” (*ver Figura 4.13*)

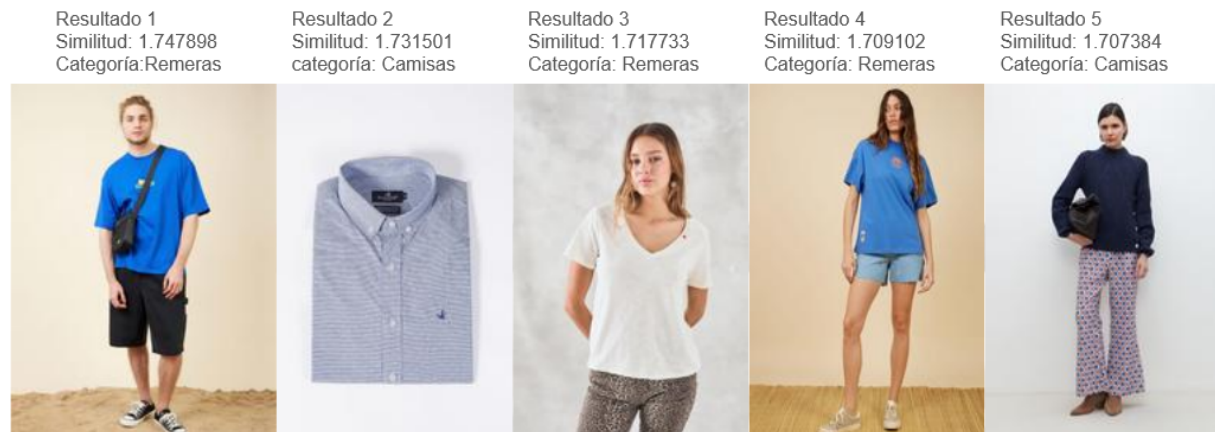


Figura 4.13: Primer test de búsqueda dentro del texto en español.

La segunda prueba fue contra la columna embedding_en. Texto ingresado: “T-Shirt white for women” (*ver Figura 4.14*)

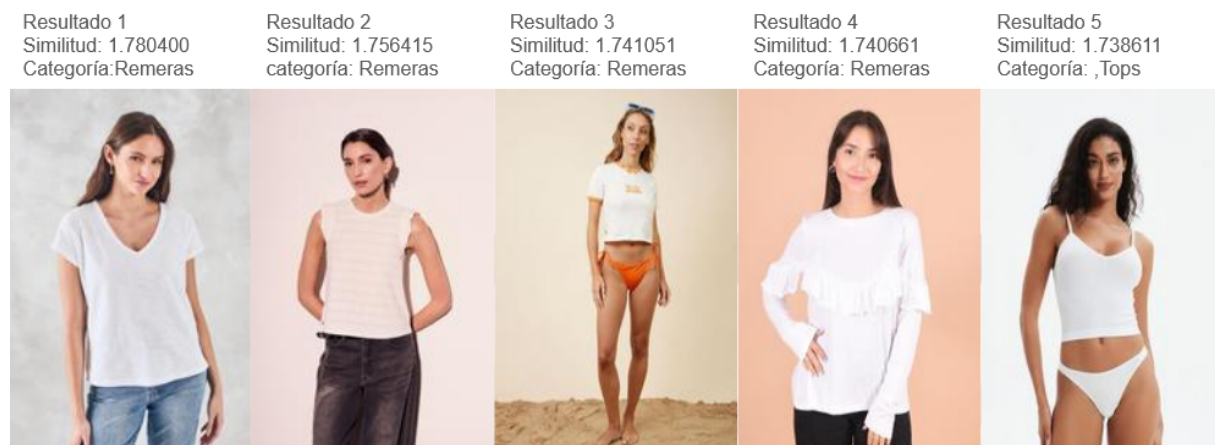


Figura 4.14: Segundo test de búsqueda dentro del texto en inglés.

La tercera prueba fue contra los embeddings de las imágenes, en la cual se usó una foto propia de la base de datos para verificar la eficacia de la búsqueda por similitud. La columna utilizada es img_embedding:

En este caso se logró acertar como imagen más precisa la que usamos de muestra y que coincide con la primera que encuentra.(*ver Figura 4.15*)

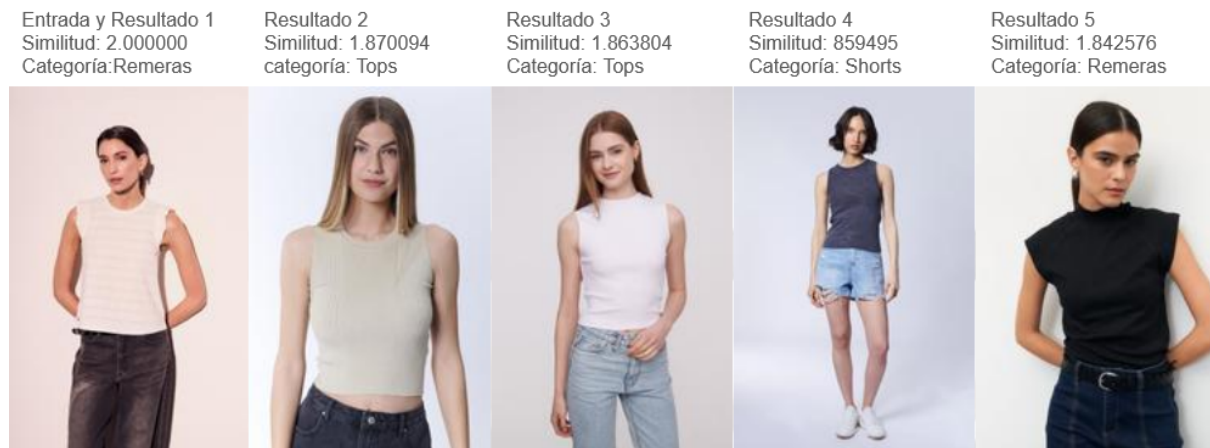


Figura 4.15: Tercer prueba contra los embeddings de las imágenes.

Lo que se observa en estas pruebas es que los aciertos difieren mucho de las categorías reales guardadas en la base. Quiere decir que no está encontrando precisamente el sexo ni la categoría de la prenda consultada. Otra observación es que en las imágenes contamos con prendas extras las cuales son almacenadas por el embedding de las imágenes cuando se ejecuta el modelo Open Fashion Clip ya que este modelo realiza una descripción de lo que ve en la imagen.

Para la siguiente prueba se utilizan las categorías almacenadas en la columna `text_for_embedding` para obtener la categoría, el género y el color de la prenda localizada. Para ello, se necesita realizar un mapeo previo que, para este caso, se realizó un mapeo manual de los géneros y de las categorías, solamente buscando formas diferentes de nombrar el sexo de una persona o el tipo de prenda a la que se puede referir.

Consulta utilizada para localizar por la categoría, género y color de los artículos:

```

1 WITH referencia AS (
2     SELECT %s::vector AS text_embedding_ref
3 )
4 SELECT
5     p.product_id ,
6     p.text_for_embedding::json ->> 'categoría' AS categoria ,
7     p.text_for_embedding::json ->> 'género' AS genero ,
8     p.text_for_embedding::json ->> 'color' AS color ,
9     p.text_for_embedding ,
10    p.img_base64 ,
11    1 - (p.embedding <#> r.text_embedding_ref) AS similitud
12 FROM public.product_embeddings p
13 JOIN referencia r ON true
14 WHERE {where_clause}
15 ORDER BY p.embedding <#> r.text_embedding_ref
16 LIMIT %s;
```

Normalización Semántica

En el caso de la cuarta prueba, se detectó que el cross-modal no realiza búsquedas precisas por la falta de información en el contexto del input del usuario. Por tal motivo, se implementaron diccionarios de normalización para garantizar consistencia:

- **Categorías:** 40+ mapeos (ej: “camisa” → “Camisas”, “jean” → “Pantalones”)
- **Géneros:** 7 mapeos (ej: “hombre” → “Caballero”, “mujer” → “Dama”)

```

1 # Mapeamos las palabras para calzarlas en las categorías normalizadas
2 map_genero = {
3     "hombre": "Caballero",
4     "caballero": "Caballero",
5     "masculino": "Caballero",
6     "mujer": "Dama",
7     "dama": "Dama",
8     "femenino": "Dama",
9     "niño": "Niño",
10    "niña": "Niña",
11    "unisex": "Unisex",
12 }
```

La quinta prueba fue ir contra los embedding de texto pero con los parámetros mapeados: “remera manga corta para muje”(ver Figura 4.16)



Figura 4.16: Quinta prueba, en este caso una búsqueda híbrida en español.

En este punto se detecta una mejoría en la similitud de las prendas elegidas, pero se encuentra un inconveniente en la obtención de los colores de las prendas, ya que en algunos casos los colores traen texto asociado, urls o codificaciones propias de la tienda, como por ejemplo «*Estampado 12*» que hacen que no se pueda encontrar la prenda asociada a un color específico. Por tal motivo se omite dicho parámetro de la consulta sql en las siguientes pruebas. (ver Figura 4.17)

```
Describe la prenda que quieres (ej: 'remera blanca para hombre'): remera manga corta para mujer
🔍 Filtros → categoría: Remeras, género: Dama
product_id categoria genero color \
0      9765  Remeras  Dama      ["Nácar"]
1     17486  Remeras  Dama      ["VARIANTE 17"]
2      9125  Remeras  Dama ["Estampado 1", "Estampado 2", "Estampado 3"]
3     17526  Remeras  Dama      ["ESTAMPADO 12"]
4     17636  Remeras  Dama      ["ESTAMPADO 6"]

similitud \
0  1.782903
1  1.757116
2  1.748779
3  1.739155
4  1.736007
```

Figura 4.17: Se encuentran textos en colores que no pueden ser identificados y que son propios de las tiendas.

4.6.1 Decisión de Arquitectura: Sistema Híbrido CatBoost + LLM

En este punto crítico del pipeline se estableció la necesidad de implementar un sistema de clasificación especializado que pudiera predecir la categoría y el género directamente desde el input del usuario. Esta decisión arquitectónica responde a dos objetivos fundamentales:

1. **Optimización de recursos computacionales:** Reservar los modelos LLM grandes (GPT-4) para tareas complejas que requieren razonamiento contextual profundo (análisis de ocasiones, generación de outfits, respuestas conversacionales), mientras que las búsquedas simples se resuelven con modelos ML locales de menor costo.
2. **Reducción de latencia y costos:** CatBoost ejecuta predicciones en <100ms sin costo de API, vs. 2-5 segundos y costos por consulta del LLM. Para consultas directas como “pantalón negro hombre”, el sistema puede clasificar y buscar sin invocar Azure OpenAI.

4.7 Generación de Mapping en categoría y en género con CatBoost

En esta etapa se entrena un modelo de clasificación basado en CatBoost [7] para predecir la categoría y el género de las prendas. En esta etapa lo que se hace es regenerar los embeddings obtenidos a partir de la columna `text_for_embedding` para regenerarlo con la categoría y el género obtenido desde el scrapping realizado a las tiendas (*Ver anexo II*). Esto permitirá mejorar los resultados del modelo sin caer el Leakage que es incluir las variables objetivo dentro del entrenamiento para que el modelo solo aprenda a repetir lo que se encuentra en ese campo, generando una percepción de un modelo optimizado.

Este modelo permite mapear de manera automática y consistente las categorías y géneros, tanto en español como en inglés, facilitando la búsqueda y clasificación en el catálogo. Esto se logra mediante la preparación de las columnas `text_for_embedding` y `text_for_embedding_en` en regeneradas nuevamente para el entrenamiento.

Lo primero que se aprecia en el análisis previo es el desbalanceo en la categoría (*ver Figura 4.18*). Para el caso del género, (*ver Figura 4.19*).

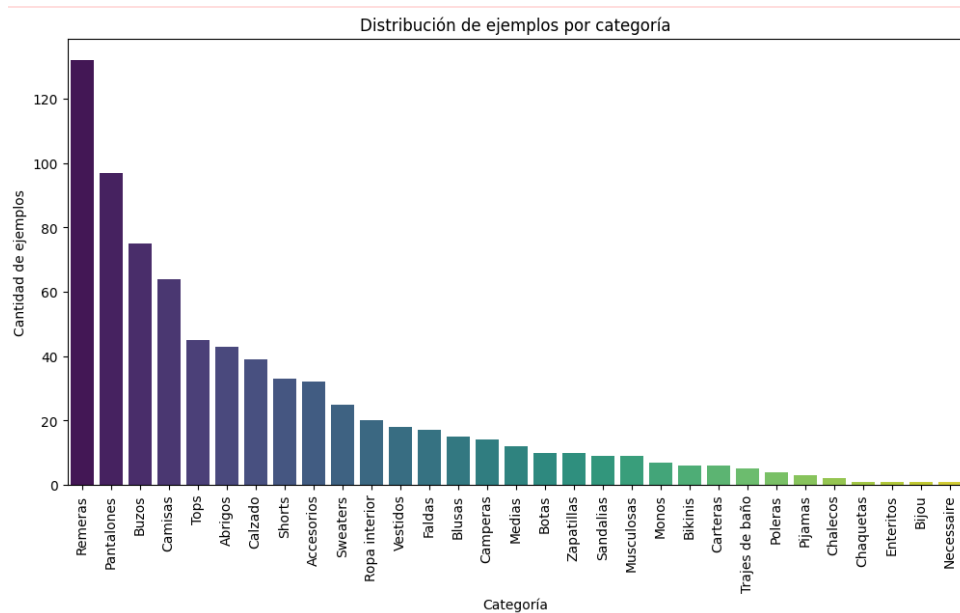


Figura 4.18: Distribución por categoría con desbalanceo, de aprecia una fuerte presencia de remeras pantalones y buzos pero pocos enteritos, bijoux y neccesaires (menos de 5).

En base a este desbalanceo en donde existen muy poca presencia de las categorías menos presentes, se opta por eliminar las categorías con menos de 5 artículos en el dataset.

```

1 # 1.4 Filtrar categorías con menos de 5 apariciones
2 # Eliminamos clases con muy pocas muestras para evitar overfitting
3 # y asegurar que haya suficientes ejemplos para train/test
4 category_counts = df_filtrado['categoria'].value_counts()
5 categories_to_keep = category_counts[category_counts >= 5].index
6 df_filtrado = df_filtrado[df_filtrado['categoria'].isin(categories_to_keep)]

```

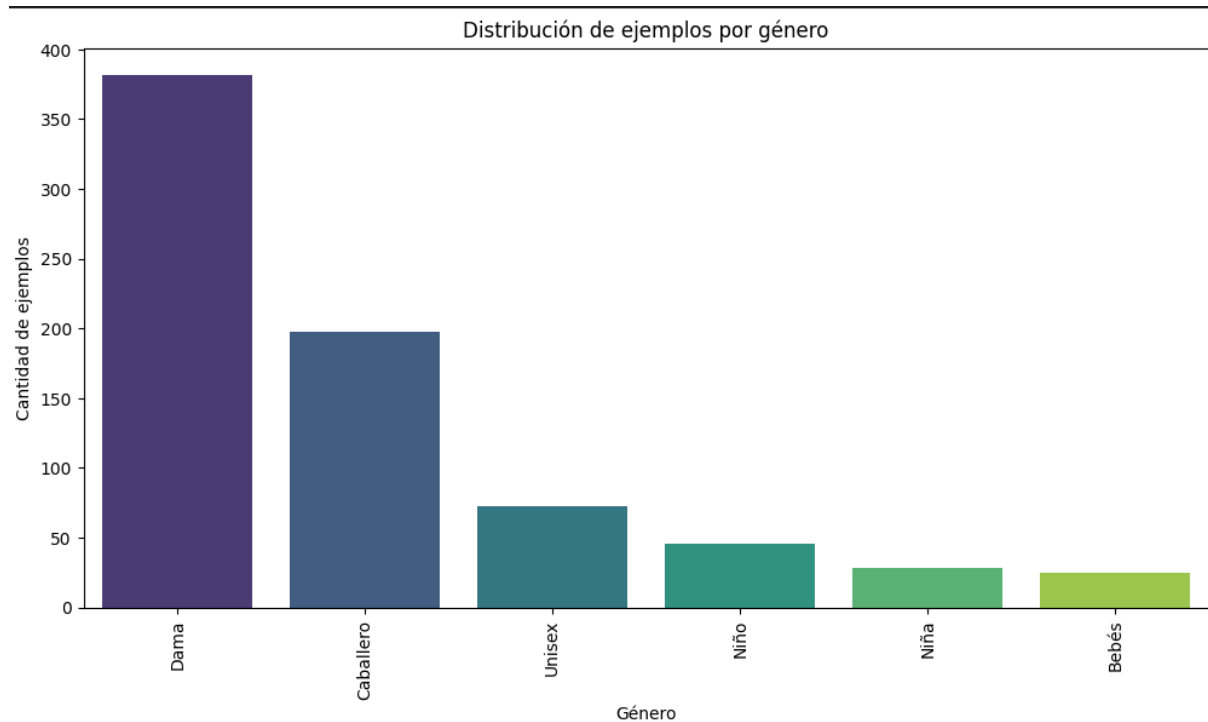


Figura 4.19: Distribución por género con alta presencia de artículos de caballeros y damas pero menos artículos de niña y bebés.

4.8 Evaluación del *Image Embedding* con Catboost

Se llevó a cabo una evaluación inicial de ingeniería de características basada en la concatenación de los tres vectores de *embedding* disponibles: el de texto en español (*embedding*), el de texto en inglés (*embedding_en*) y el de imagen (*img_embedding*). El objetivo era potenciar la clasificación aprovechando la información visual del producto.

4.8.1 Problema de Ruido Contextual

Tras el entrenamiento y posterior validación del modelo en un entorno de prueba con datos no utilizados, se detectó un problema de **sobreajuste al ruido contextual** introducido por la característica *img_embedding*.

El clasificador demostró ser incapaz de aislar el producto principal de los elementos visuales secundarios presentes en la imagen (e.g., el fondo, o prendas adicionales vestidas por la modelo). Específicamente, en pruebas con productos como un pantalón blanco, el modelo se concentraba excesivamente en el color o patrón de una prenda secundaria (un buzo con rayas rojas) visible en la imagen de cuerpo completo. (ver *Figura 4.20*).



Figura 4.20: La búsqueda fue realizada con *Pantalón rojo dama* mostrando categoría y género correcto, pero el color no es el esperado en la prenda buscada. Este error se debe al ruido contextual introducido por el *Image Embedding*.

Este fenómeno generó errores críticos de clasificación, haciendo que el modelo predijera categorías y atributos de color que estaban **fuera de los parámetros del producto principal** que debían ser clasificados.

4.8.2 Decisión Metodológica

Dado que el *embedding* de imagen introducía ruido que degradaba la precisión de la clasificación, se determinó excluir la característica *img_embedding* del conjunto final. La estrategia se centró exclusivamente en la concatenación de los **embeddings** de texto (*embedding* y *embedding_en*), ya que estos vectores proporcionaron una representación **semántica pura** del producto.

4.9 Entrenamientos por peso en datos desbalanceados

4.9.1 Categoría

En base a que la aplicación de pesos diferentes realizada de forma manual puede resultar ineficiente, se optó por utilizar la librería *sklearn.utils.class_weight*. Esta librería permite calcular

automáticamente los pesos inversamente proporcionales a la frecuencia de cada clase. Esto se implementó con la siguiente instrucción:

```

1 from sklearn.utils.class_weight import compute_class_weight
2
3 # Calcular class weights para balanceo
4 def calculate_class_weights(y):
5     class_weights = class_weight.compute_class_weight(
6         class_weight='balanced',
7         classes=np.unique(y),
8         y=y
9     )
10    return dict(zip(np.unique(y), class_weights))

```

El cálculo de los pesos de clase se basa en la frecuencia de cada clase dentro del conjunto de entrenamiento. La fórmula utilizada es:

$$w_i = \frac{n}{k \cdot n_i}$$

donde:

- w_i es el peso asignado a la clase i .
- n es el número total de muestras.
- k es el número de clases.
- n_i es el número de muestras de la clase i .

No solo se debió aplicar la técnica de pesos en este trabajo, sino que también se tuvo que eliminar datos con menos de 5 apariciones para evitar categorías con poca presencia.

```

1 # 1.4 Filtrar categorías con menos de 5 apariciones
2 # Eliminamos clases con muy pocas muestras para evitar overfitting
3 # y asegurar que haya suficientes ejemplos para train/test
4 category_counts = df_filtrado['categoria'].value_counts()
5 categories_to_keep = category_counts[category_counts >= 5].index
6 df_filtrado = df_filtrado[df_filtrado['categoria'].isin(categories_to_keep)
7     ↪ ]
8
9 print(f"Dataset después de filtrar categorías: {len(df_filtrado)} filas")
10 print(f"Categorías restantes: {df_filtrado['categoria'].nunique()}")

```

Para este caso no se optó por aplicar oversampling ya que dicha técnica generaría una cantidad grande de datos sintéticos lo que provocaría que el modelo se entrene con más del doble con datos sintéticos y podría generar problemas de overfitting en el entrenamiento y a la hora de montarlo en producción no presentar la misma eficacia.

Adicionalmente, se aplican pesos de clase (class weights) para balancear las categorías minoritarias. En este experimento se utiliza únicamente el 20 % del dataset (149 de 746 artículos) como conjunto de entrenamiento, ajustando el número de iteraciones a 1000. Los resultados obtenidos son:

Cuadro 4.4: Reporte de clasificación del modelo en la categoría

Clase	Precisión	Recall	F1-score	Soporte
Abrigos	0.90	1.00	0.95	9
Accesorios	0.88	1.00	0.93	7
Bikinis	0.00	0.00	0.00	1
Blusas	1.00	0.67	0.80	3
Botas	1.00	1.00	1.00	2
Buzos	0.82	0.93	0.88	15
Calzado	0.78	1.00	0.88	7
Camisas	0.85	0.85	0.85	13
Camperas	1.00	0.33	0.50	3
Carteras	0.00	0.00	0.00	1
Faldas	1.00	1.00	1.00	3
Medias	1.00	1.00	1.00	2
Monos	1.00	1.00	1.00	1
Pantalones	1.00	1.00	1.00	20
Remeras	0.77	0.89	0.83	27
Ropa interior	0.75	0.75	0.75	4
Sandalias	1.00	1.00	1.00	2
Shorts	1.00	1.00	1.00	7
Sweaters	1.00	0.20	0.33	5
Tops	0.83	1.00	0.91	10
Trajes de baño	0.00	0.00	0.00	1
Vestidos	1.00	1.00	1.00	4
Zapatillas	1.00	0.33	0.50	3
Exactitud	0.87 (150)			
Macro avg	0.81	0.74	0.74	150
Weighted avg	0.87	0.87	0.85	150

Revisando un poco más los detalles en la matriz de confusión, se encuentra que algunas de las categorías que más se confunden son los buzos y los pantalones(*ver Figura 4.21*).

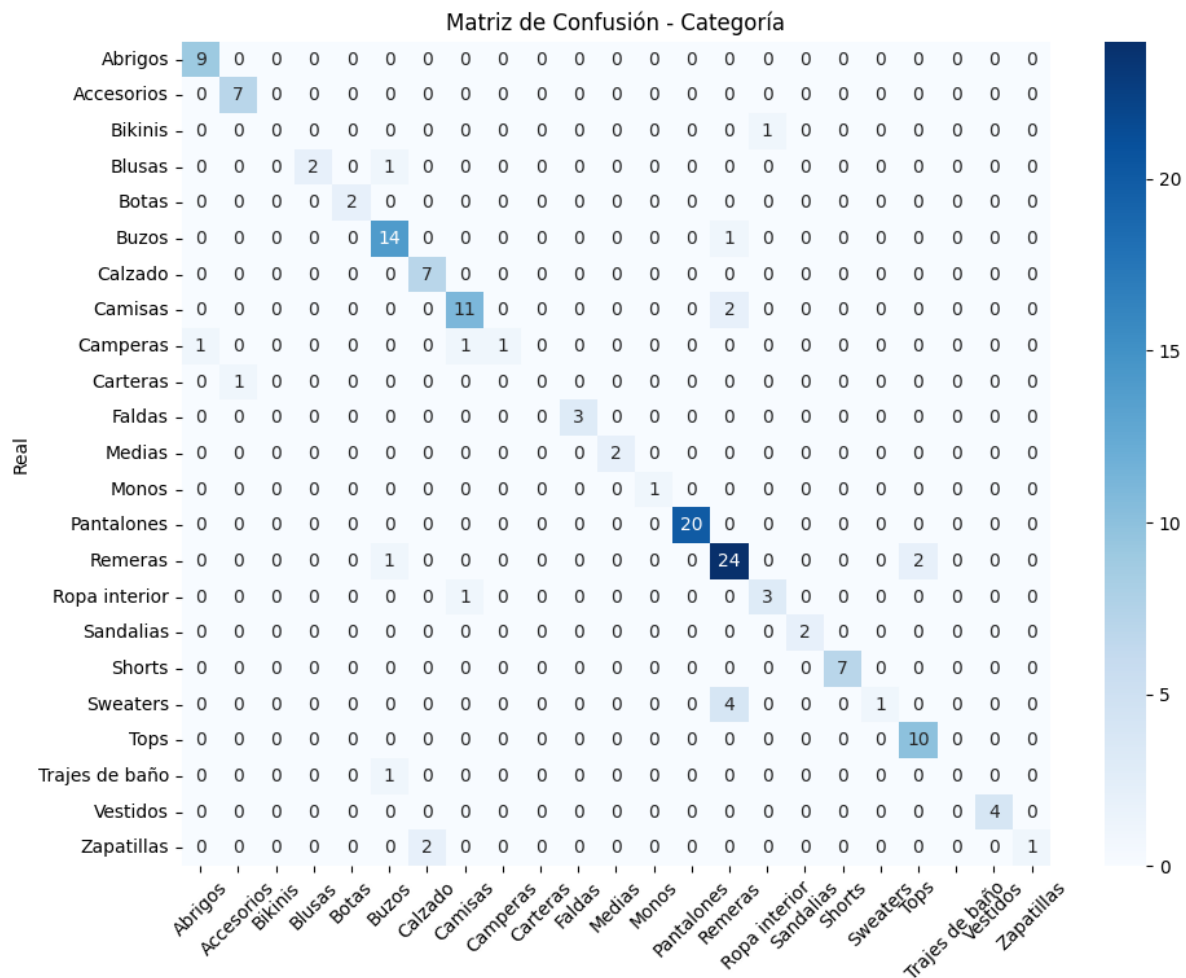


Figura 4.21: Matriz de confusión obtenida luego de aplicar diferentes pesos a la categoría.

4.9.2 Género

En el caso del género, se aplica `class_weight` para aumentar el peso de las clases con poca presencia antes de entrenar el modelo y obtener mejores resultados.

Cuadro 4.5: Reporte de clasificación del modelo en el género

Clase	Precisión	Recall	F1-score	Soporte
Bebés	1.00	0.60	0.75	5
Caballero	0.93	0.95	0.94	39
Dama	0.93	1.00	0.96	76
Niña	0.80	0.67	0.73	6
Niño	1.00	0.89	0.94	9
Unisex	0.92	0.73	0.81	15
Exactitud	0.93 (150)			
Macro avg	0.93	0.81	0.86	150
Weighted avg	0.93	0.93	0.92	150

Estos mismos resultados, pueden apreciarse en la matriz de confusión asociada a dicha variable categórica (ver Figura 4.22).

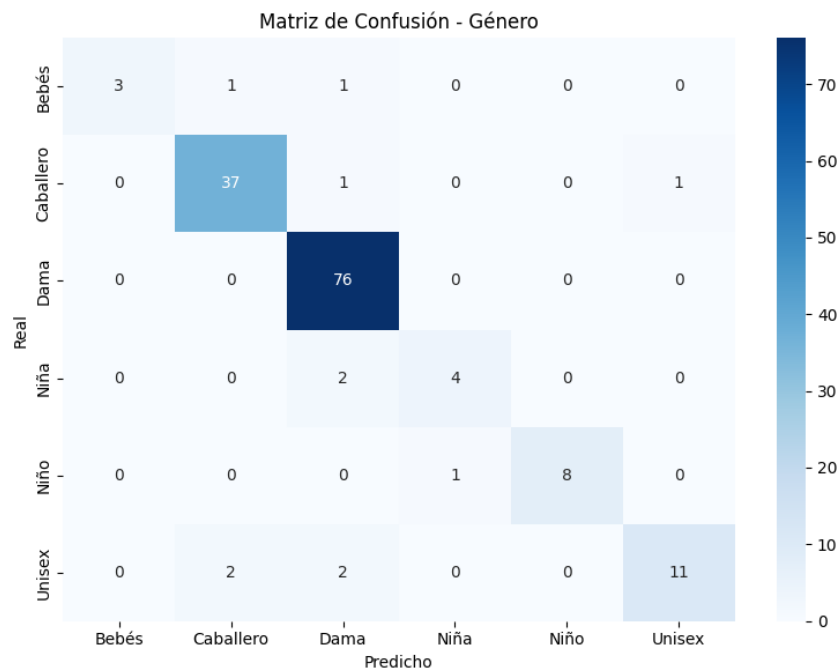


Figura 4.22: Matriz de confusión obtenida luego de aplicar diferentes pesos al género.

En uno de los primeros entrenamientos de forma manual se le asignó un peso a los géneros con poca presencia, esto se realizó antes de utilizar la librería específica para realizar dicho cálculo. De todas formas se mantiene dicho gráfico para visualizar la evolución de las pruebas hasta llegar al modelo obtenido. En este caso el entrenamiento fue sobre el total de las filas con 300 iteraciones (Ver ANEXO I). En el gráfico se puede visualizar el comportamiento con un leve sobreajuste en el mismo (ver Figura 4.23): la precisión en entrenamiento continúa creciendo hasta alcanzar valores cercanos a 1.0, mientras que la precisión en validación se estabiliza en torno a 0.85 a partir de la iteración 150. Esto sugiere que entrenar más allá de ese punto no aporta mejoras significativas y que un *early stopping* podría ser una estrategia adecuada para evitar entrenamientos innecesariamente largos.

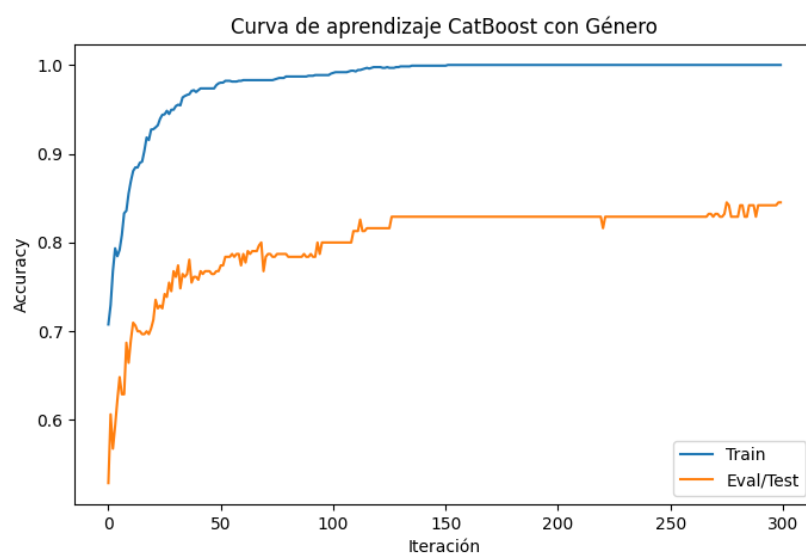


Figura 4.23: Curva de aprendizaje del modelo CatBoost para la clasificación de género.

Para asegurarse de que el modelo no esté sobreajustando y obtener mejores resultados de precisión, recall y f1-score, se aplica cross-validation estratificado, haciendo solamente 150 iteraciones por lo ya visto en la curva de aprendizaje en la cual no difiere de los resultados

anteriores.

Una de las opciones definidas para evitar el overfitting fue la detección temprana de falta de mejoras en las últimas 50 iteraciones que se realizan durante el entrenamiento aplicando *early_stopping_rounds=50*. De esta forma se evita el tener que identificar el punto de entrenamiento donde comienza a existir overfitting y entrenar innecesariamente.

```

1 # 5.1 Configuración del modelo CatBoost para CATEGORÍA
2 model_cat = CatBoostClassifier(
3     iterations=1000,                # Número máximo de árboles
4     learning_rate=0.1,              # Tasa de aprendizaje
5     depth=6,                        # Profundidad máxima de cada árbol
6     loss_function='MultiClass',    # Función de pérdida para
7                                     ↪ clasificación multiclase
8     random_state=42,                # Semilla para reproducibilidad
9     verbose=100,                    # Mostrar progreso cada 100
10                                     ↪ iteraciones
11     class_weights=class_weights_cat, # Pesos de clase para balanceo
12     early_stopping_rounds=50        # Detener si no hay mejora en 50
13                                     ↪ iteraciones
14 )

```

Listing 4.2: Configuración del modelo CatBoost para CATEGORÍA

4.9.3 Resumen de los entrenamientos

La evaluación de los modelos de clasificación (*Categoría* y *Género*), entrenados con los vectores *embeddings* y *embeddings_en* concatenados generando uno nuevo de dimensión 1024, confirma una alta eficacia y estabilidad, superando el estándar para tareas de clasificación multiclase de alta complejidad.

Previamente se había intentado concatenar los tres embeddings existentes para lograr una mejor predicción en las búsquedas, pero al momento de probar el modelo en el servicio con datos no utilizados en el entrenamiento se detectó que el modelo se desviaba de las categorías mencionadas por el cliente y se concentraba en el color de otra prenda, haciendo que el modelo muestre prendas fuera de los parámetros esperados.

Como dato importante en este entrenamiento se utilizó la información de la columna *text_for_embedding* que se había generado, eliminando las variables objetivo dentro de la misma y sustituyendo por la categoría y el género obtenidos del scrapping realizado a las tiendas.

Ejemplo de *text_for_embedding* previo a la generación del vector:

```

1 {
2     'color': ['Lila Pastel'],
3     'nombre': 'TOP TRIANGULO FIJO DALIA',
4     'genero': 'female',
5     'categoria': 'Bikinis y mallas',
6     'descripcion': 'Parte superior de bikini de triángulos fijos con breteles
7                     ↪ finos regulables. \
8                     Diseño de estampa de flores bicolor. Cuenta con forro interior y copas
9                     ↪ desmontables. \
10                    Composición: 85% Poliamida, 15% Elastano.'
11 }

```

Listing 4.3: Ejemplo de estructura de datos de prenda

4.10 Métricas de Robustez y Generalización

Para demostrar la robustez de los clasificadores CatBoost ante el riesgo de sobreajuste, se empleó la Validación Cruzada Estratificada (5-folds). Los resultados, centrados en el F1-Score Ponderado, los siguientes cuadros.

Cuadro 4.6: Métricas detalladas del modelo — Categoría

Métrica	Valor	Comentario
Exactitud (Accuracy)	0.8733	Porcentaje total de predicciones correctas.
Precisión (Precision)	0.8684	Proporción de predicciones correctas entre las positivas.
Exhaustividad (Recall)	0.8733	Capacidad del modelo para detectar correctamente las clases reales.
Puntaje F1 (F1-Score)	0.8521	Media armónica entre precisión y exhaustividad; mide el equilibrio entre ambas.

Cuadro 4.7: Métricas detalladas del modelo — Género

Métrica	Valor	Comentario
Exactitud (Accuracy)	0.9267	Alto porcentaje de aciertos generales en la clasificación.
Precisión (Precision)	0.9271	Indica pocas falsas predicciones positivas.
Exhaustividad (Recall)	0.9267	Mide qué tan bien se detectan las clases verdaderas.
Puntaje F1 (F1-Score)	0.9230	Confirma el equilibrio entre precisión y recall, con un rendimiento muy estable.

El **Modelo de Categoría** exhibe un rendimiento alto con un **F1-Score de 85.21 %**, lo que prueba que el modelo generaliza consistentemente a diferentes subconjuntos de datos. El **Modelo de Género** demuestra una **confiabilidad superior (F1: 0.9251)**, lo que valida la robustez de la representación semántica de los productos.

Se puede identificar que, en el caso del género, se obtiene un *92.30 %*

En ambos casos, se aplicó la detección temprana de sobreajuste (overfitting) para evitarlo e interrumpir el entrenamiento:

```

1 # 6.2 Entrenar el modelo con conjunto de validación
2 # eval_set=(X_test_gen, y_test_gen) permite monitorear el overfitting
3 # El modelo guarda internamente el mejor modelo según el eval_set
4 model_gen.fit(
5     X_train_gen, y_train_gen,
6     eval_set=(X_test_gen, y_test_gen),
7     verbose=100
8 )

```

Listing 4.4: Entrenamiento del modelo con conjunto de validación

4.11 Interfaz de usuario y complementos

Posteriormente a la limpieza de los datos, el *feature engineering* y el entrenamiento de dos modelos de *machine learning* para la predicción de la categoría y el género en dos idiomas diferentes, contando además con los *embeddings* de las características de las prendas y de las imágenes, se procede a generar la aplicación web que servirá como punto de conexión con los usuarios interesados en conocer las prendas disponibles en plaza según sus gustos.

El sistema web desarrollado integra un diseño modular que facilita la navegación y visualización de resultados. Los componentes principales de la interfaz incluyen el sistema de autenticación, el menú de navegación y los controles de configuración.

4.11.1 Sistema de inicio de sesión

El diseño web incluye un sistema de inicio de sesión (ver figura 4.24), en el cual el usuario puede registrarse completando un formulario que recopila información relevante. Para evitar el sobredimensionamiento del proyecto, se optó por solicitar únicamente los siguientes datos: user_id, nombre, apellido, foto de perfil y contraseña.

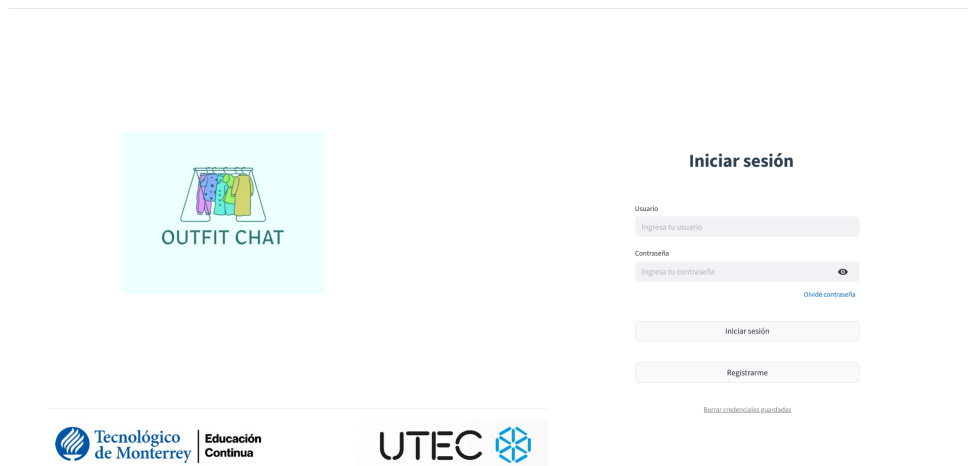


Figura 4.24: Interfaz del sistema de inicio de sesión del frontend desarrollado en Streamlit.

4.11.2 Sidebar de configuración y ajustes

En el mismo sidebar, también se visualizan los ajustes del sistema para manipular su comportamiento y visualizar la predicción de lo que el usuario consulta con lo que se encuentra en la base de datos (ver figura 4.25). Esto permite identificar en qué aspectos los modelos de CatBoost están fallando en base a lo que el usuario ingresa. Esto ayudó a detectar fallas al recurrir a otra comparación, que es el embedding de la imagen, generando una búsqueda híbrida. Más abajo se encuentra la leyenda con un semáforo según la similitud que el modelo encontró entre la consulta del usuario y la base de datos.



Figura 4.25: Sidebar con los ajustes y las leyendas para visualizar el comportamiento de las búsquedas de los modelos.

4.11.3 Menú de navegación del sistema

La aplicación cuenta con un **menú de navegación** en el sidebar que proporciona acceso a las diferentes secciones del sistema web (ver figura 4.26). Este menú permite al usuario realizar múltiples acciones: desde interactuar con el asistente de moda hasta auditar el rendimiento del sistema multi-agente, consultar la documentación técnica y gestionar configuraciones avanzadas.

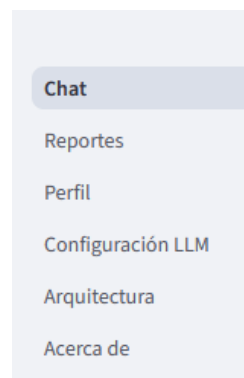


Figura 4.26: Menú lateral con las pestañas de navegación del sistema.

Secciones disponibles en el menú

- **Chat:** Interfaz principal donde el usuario ingresa consultas en lenguaje natural o adjunta imágenes para búsqueda visual de productos. Esta es la pestaña predeterminada del

sistema, diseñada para la interacción conversacional con el asistente inteligente.

- **Reportes:** Dashboard de análisis y auditoría del sistema que muestra estadísticas detalladas: actividad de usuarios, métricas de rendimiento de agentes LLM, costos de operación (tokens consumidos y USD), distribución de búsquedas por categoría/género y gráficos de evolución temporal. Orientado a la evaluación académica y al monitoreo operativo.
- **Perfil:** Panel de configuración personal del usuario que permite modificar los datos de la cuenta (nombre, apellido, contraseña), actualizar la foto de perfil y revisar el historial de interacciones con el sistema.
- **Configuración LLM:** Sección administrativa para ajustar parámetros del sistema multi-agente. Incluye dos subsecciones principales:
 - *Gestión de Prompts:* Editor en tiempo real para los 13 prompts almacenados en base de datos, distribuidos entre 6 agentes LLM (AnalizadorConsulta, Coordinador, ImageAnalyzer, EspecialistaOutfits, RefinadorConsulta, PresentadorResultados).
 - *Calibración de Temperatura:* Control de la creatividad/determinismo de cada agente (rango 0.0-1.0), con valores recomendados preconfigurados según la función de cada agente.
- **Arquitectura:** Documentación técnica, visual e interactiva del sistema. Presenta dos vistas:
 - *Diagrama Interactivo:* Visualización HTML con las 9 fases del flujo de procesamiento, desde la entrada del usuario hasta la presentación de resultados, incluyendo el sistema de auditoría LLM.
 - *README:* Documentación completa del proyecto (versión actual: v2.7.1) con historial de versiones, guía de instalación, estructura del proyecto y ejemplos de uso.
- **Acerca de:** Información general del sistema StyleDesigner: versión actual (v2.7), características principales, tecnologías utilizadas (Streamlit, Azure OpenAI, PostgreSQL, CatBoost), lista de los 7 agentes especializados (AnalizadorConsulta, ImageAnalyzer, ClasificadorCatBoost, BuscadorEmbeddings, PresentadorResultados, EspecialistaOutfits, RefinadorConsulta) y datos del proyecto académico (UTEC-TEC).

4.12 Diagrama macro del sistema

El sistema realiza una serie de pasos según el comportamiento del usuario. En este diagrama se muestra el comportamiento a nivel macro para entender la lógica principal (para ver la lógica completa, ir al ANEXO IV y ANEXO V) (ver figura 4.27).

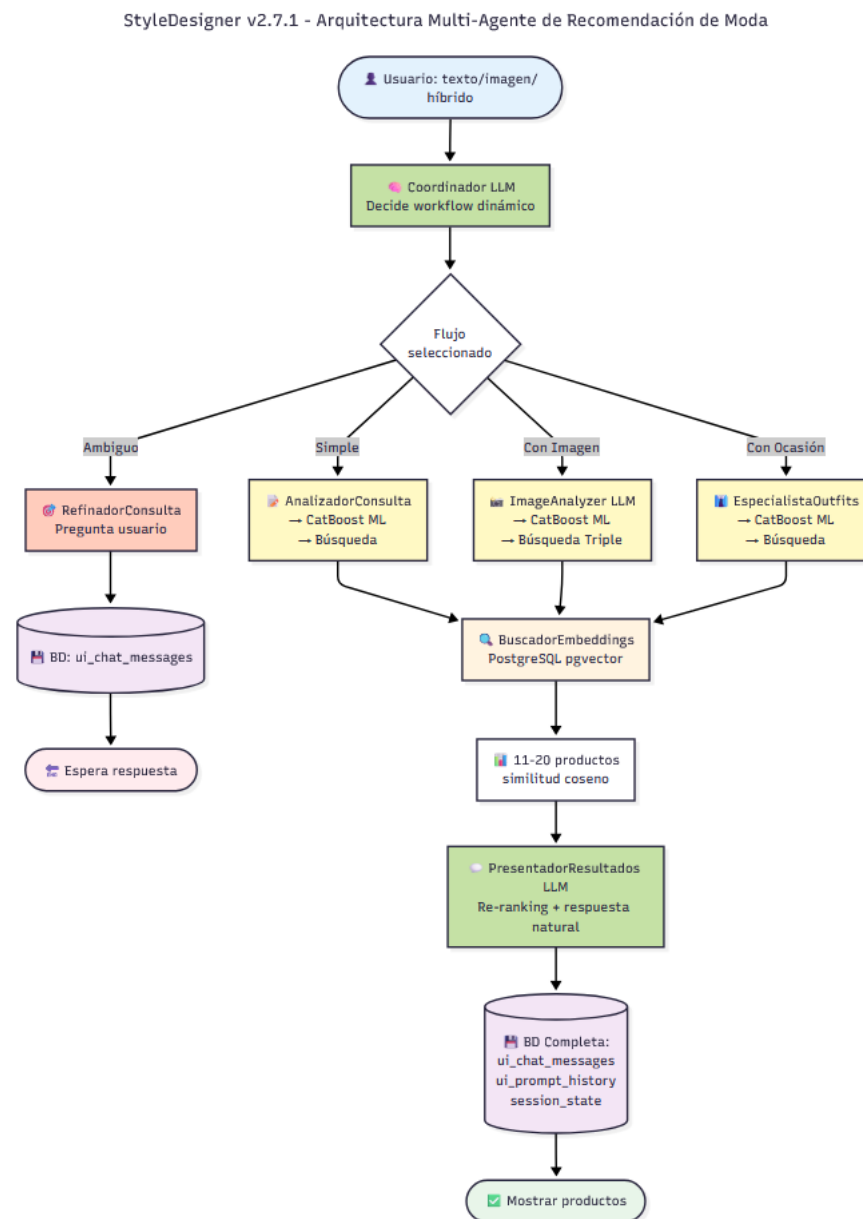


Figura 4.27: Diagrama macro del sistema recomendador de prendas.

4.13 Stack tecnológico

El sistema integra 7 agentes especializados coordinados por un orquestador custom (FashionSearchCoordinator). A continuación se listan las tecnologías principales empleadas y su rol en la solución:

Python 3.12+ Lenguaje de implementación del sistema

Streamlit Framework web para la interfaz de usuario interactiva

PostgreSQL + pgvector Base de datos relacional con soporte vectorial para embeddings

Azure OpenAI GPT-4 Modelo de lenguaje para 5 agentes LLM (AnalizadorConsulta, ImageAnalyzer, EspecialistaOutfits, PresentadorResultados, RefinadorConsulta)

LangChain (parcial) Wrapper para invocar Azure OpenAI y con el agente image_analyzer con la función HumanMessage

CatBoost Modelos de Machine Learning para clasificación de categoría y género

OpenFashion CLIP Embeddings visuales de 512 dimensiones para análisis de imágenes

Sentence Transformers Embeddings textuales de 512 dimensiones para búsqueda semántica

JWT + localStorage Sistema de autenticación con persistencia de sesión

Ventajas de uso del Coordinador custom: Se implementa un coordinador custom (FashionSearchCoordinator) que:

- Decide dinámicamente qué agentes ejecutar y en qué orden
- Maneja 7 patrones de workflow según tipo de consulta
- Integra lógica híbrida (LLM + reglas heurísticas)
- Mantiene fallbacks para funcionar sin LLM

4.14 Fundamentos teóricos de sistemas multi-agente

Cuando hablamos de sistemas multiagente, nos referimos a un grupo de especialistas que cumplen un rol específico para lograr un objetivo común. Cada agente puede tener su propio prompt, LLM, código o herramientas para obtener información de una base de datos, por ejemplo.

4.14.1 Clasificación de agentes

Los agentes se clasifican según su arquitectura interna en tres categorías principales:

- **Reactivos:** Responden a estímulos del entorno sin mantener estado interno
- **Deliberativos:** Mantienen un modelo del mundo y planifican acciones
- **Híbridos:** Combinan comportamiento reactivo con razonamiento deliberativo

El sistema StyleDesigner implementa agentes híbridos que combinan reglas heurísticas (componente reactivo) con razonamiento LLM (componente deliberativo).

4.15 Arquitectura de coordinación multi-agente con LangChain

El sistema utiliza LangChain selectivamente para la conexión con Azure OpenAI, implementando coordinación propia para maximizar flexibilidad y auditoría.

4.15.1 Componentes de LangChain utilizados

El sistema integra únicamente tres componentes específicos de LangChain:

1. AzureChatOpenAI: Cliente para conexión con GPT-4o-mini y GPT-4 Vision
2. HumanMessage: Construcción de mensajes multimodales (texto + imagen)
3. `.invoke()`: Invocación sincrónica con control de parámetros

```

1 from langchain_openai import AzureChatOpenAI
2 from langchain_core.messages import HumanMessage
3
4 # 1. Cliente LLM con Azure OpenAI
5 def create_azure_llm(agent_name='Coordinador', temperature=0.1):
6     """Crea instancia de LLM con configuracion por agente."""
7     llm = AzureChatOpenAI(

```

```

8         azure_endpoint=os.getenv("AZURE_OPENAI_ENDPOINT"),
9         openai_api_key=os.getenv("AZURE_OPENAI_API_KEY"),
10        openai_api_version="2024-02-15-preview",
11        deployment_name="gpt-4o-mini",
12        temperature=temperature,
13        max_retries=3,
14        timeout=30
15    )
16    return llm
17
18 # 2. Mensajes multimodales para GPT-4 Vision
19 message = HumanMessage(
20     content=[
21         {"type": "text", "text": "Describe esta prenda en detalle:"},
22         {"type": "image_url", "image_url": {
23             "url": f"data:image/jpeg;base64,{image_b64}"
24         }}
25     ]
26 )
27
28 # 3. Invocacion directa del LLM
29 response = llm.invoke([message], max_tokens=150)
30 description = response.content

```

Listing 4.5: Componentes de LangChain utilizados en StyleDesigner

La función ajusta temperatura por agente (baja=determinístico, alta=creativo), los mensajes combinan texto e imágenes en base64, y invoke() ejecuta llamadas sincrónicas con límite de tokens.

4.15.2 Justificación del uso limitado de LangChain

La decisión de implementar una arquitectura propia de coordinación en lugar de utilizar el framework completo de LangChain se fundamenta en cinco requisitos críticos:

- **Mayor control:** La orquestación manual permite implementar flujos dinámicos adaptativos (7 patrones de workflow identificados) que serían complejos de lograr con AgentExecutor
- **Fallback robusto:** El sistema puede operar completamente sin LLM mediante un flujo estático predefinido, lo que no es posible con la arquitectura estándar de LangChain
- **Optimización de costos:** La ejecución condicional de agentes según tipo de consulta reduce llamadas LLM innecesarias
- **Auditoría granular:** Logging personalizado de cada paso con métricas detalladas de tokens y costos en PostgreSQL
- **Configurabilidad sin redeploy:** Prompts editables en base de datos (tabla ui_agent_prompts) sin necesidad de modificar código

4.16 Arquitectura propia de coordinación

El sistema implementa cuatro componentes principales que reemplazan las abstracciones estándar de LangChain:

1. Coordinador manual El FashionSearchCoordinator orquesta la ejecución de 7 agentes especializados mediante un diccionario Python y ejecución secuencial controlada:

```

1 class FashionSearchCoordinator:
2     """Orquestador manual de agentes (NO AgentExecutor)."""
3
4     def __init__(self, agents, db_manager=None, llm=None):
5         self.agents = agents # Diccionario de 7 agentes especializados
6         self.db_manager = db_manager
7         self.llm = llm # LLM directo, NO agent_executor
8
9         # Flujo estatico de fallback (sin LLM)
10        self.default_workflow = [
11            'analizar_consulta',
12            'clasificar_catboost',
13            'ejecutar_busqueda',
14            'presentar_resultados'
15        ]
16
17    def process_query(self, user_input, llm_enabled=True):
18        """Procesa consulta con flujo dinamico o estatico."""
19        context = {'user_input': user_input}
20
21        # Decidir flujo dinamicamente con LLM o usar estatico
22        if llm_enabled and self.llm:
23            workflow = self._decide_workflow_with_llm(context)
24        else:
25            workflow = self.default_workflow
26
27        # Ejecutar agentes manualmente (NO agent_executor.invoke())
28        for step in workflow:
29            if step == 'analizar_consulta':
30                context = self.agents['analizador_consulta'].execute(
31                    ↪ context)
32            elif step == 'clasificar_catboost':
33                context = self.agents['clasificador_catboost'].execute(
34                    ↪ context)
35            elif step == 'ejecutar_busqueda':
36                context = self.agents['buscador_embeddings'].execute(
37                    ↪ context)
38            elif step == 'presentar_resultados':
39                context = self.agents['presentador_resultados'].execute(
40                    ↪ context)
41            # ... mas pasos
42
43        return context
44
45    def _decide_workflow_with_llm(self, context):
46        """LLM decide dinamicamente que agentes ejecutar."""
47        decision_prompt = f"""
48        Analiza esta consulta: {context['user_input']}
49        Decide que agentes ejecutar: [analizar_consulta, analizar_imagen,
50        clasificar_catboost, ejecutar_busqueda, especialista_outfits,
51        presentar_resultados]
52        Responde con lista JSON.
53        """
54        response = self.llm.invoke([HumanMessage(content=decision_prompt)])
55        workflow = json.loads(response.content)
56        return workflow

```

Listing 4.6: Arquitectura del coordinador manual

2. Memoria conversacional con Streamlit + PostgreSQL En lugar de ConversationBufferMemory de LangChain, el sistema utiliza:

1. **Memoria en sesión:** st.session_state de Streamlit para acceso instantáneo durante la sesión activa
2. **Persistencia:** Base de datos PostgreSQL para historial a largo plazo y análisis

```

1 import streamlit as st
2
3 # Inicializar memoria en session_state (NO ConversationBufferMemory)
4 if 'conversation_history' not in st.session_state:
5     st.session_state.conversation_history = []
6
7 # Agregar interaccion (3-tupla: user_msg, assistant_msg, images)
8 st.session_state.conversation_history.append(
9     (user_input, assistant_response, images_base64)
10 )
11
12 # Ventana deslizable automatica (limitar a 10 intercambios)
13 if len(st.session_state.conversation_history) > 10:
14     st.session_state.conversation_history = \
15         st.session_state.conversation_history[-8:]
16
17 # Persistir en PostgreSQL (auditoria completa)
18 db_manager.save_message(
19     conversation_id=conversation_id,
20     user_id=user_id,
21     message_type='assistant',
22     content=final_response,
23     agent_context={
24         'agents_used': agents_executed,
25         'processing_time_ms': processing_time,
26         'catboost_predictions': catboost_results,
27         'search_results': products_df.to_dict()
28     }
29 )

```

Listing 4.7: Gestión de memoria conversacional propia

3. Prompts configurables en base de datos Los prompts de los agentes se almacenan en PostgreSQL (tabla ui_agent_prompts) en lugar de estar hardcodedos en el código:

```

1 # Cargar prompt desde PostgreSQL (NO ChatPromptTemplate)
2 prompt = db_manager.get_agent_prompt(
3     agent_name='Coordinador',
4     prompt_type='system',
5     prompt_key='coordination_prompt'
6 )
7
8 # Ejemplo de prompt almacenado en BD:
9 # agent_name: 'Coordinador'
10 # prompt_type: 'system'
11 # prompt_key: 'coordination_prompt'
12 # prompt_content: "Eres el coordinador inteligente del sistema..."
13 # temperature: 0.1
14
15 # Invocar LLM con prompt desde BD
16 response = llm.invoke([

```

```

17 SystemMessage( content=prompt[ 'prompt_content' ] ) ,
18 HumanMessage( content=user_input )
19 ])

```

Listing 4.8: Carga de prompts desde base de datos

Ventajas de prompts en base de datos:

- Edición sin redeploy mediante interfaz web (sección Configuración LLM)
- Versionado automático de cambios en prompts
- A/B testing de diferentes prompts sin modificar código
- Temperatura configurable por agente

4. Logging LLM personalizado El sistema implementa auditoría completa de interacciones LLM en la tabla `ui_prompt_history`:

```

1 def invoke_llm_with_logging(llm, prompt, agent_name, user_id,
2                             conversation_id):
3     """Invoca LLM y registra tokens/costos en BD."""
4
5     # Invocar LLM
6     response = llm.invoke([HumanMessage(content=prompt)])
7
8     # Calcular tokens (estimacion: palabras * 1.3)
9     input_tokens = len(prompt.split()) * 1.3
10    output_tokens = len(response.content.split()) * 1.3
11
12    # Calcular costos (precios Azure OpenAI)
13    cost_input = (input_tokens / 1000) * 0.0025 # $0.0025/1K tokens
14    cost_output = (output_tokens / 1000) * 0.010 # $0.010/1K tokens
15    cost_total_usd = cost_input + cost_output
16
17    # Registrar en PostgreSQL
18    LLMLogger.log_llm_call(
19        user_id=user_id,
20        conversation_id=conversation_id,
21        agent_name=agent_name,
22        prompt_type='user',
23        user_prompt=prompt,
24        system_response=response.content,
25        input_tokens=int(input_tokens),
26        output_tokens=int(output_tokens),
27        estimated_cost_usd=cost_total_usd
28    )
29
30    return response

```

Listing 4.9: Sistema de logging LLM personalizado

Métricas registradas en base de datos:

- Tokens de entrada/salida por agente
- Costo estimado en USD por llamada
- Tiempo de procesamiento
- Prompts utilizados (versionados)
- Contexto completo de la conversación

4.16.1 Flujos de ejecución: dinámico vs estático

El sistema implementa dos modos de operación según disponibilidad de LLM:

Modo inteligente (LLM habilitado) El Coordinador decide dinámicamente qué agentes ejecutar basándose en análisis LLM del contexto:

1. Analiza consulta con LLM para determinar tipo (texto/imagen/híbrido)
2. Decide workflow adaptativo (7 patrones identificados)
3. Ejecuta solo agentes necesarios (optimización de costos)
4. Registra decisiones en BD para análisis

Ejemplo de flujo dinámico:

Usuario: "Busco vestido rojo para fiesta"

Coordinador LLM decide: [analizar_consulta, clasificar_catboost,
ejecutar_búsqueda, especialista_outfits,
presentar_resultados]

Resultado: 5 agentes ejecutados (omite analizar_imagen, refinador_consulta)

Modo fallback (sin LLM) Flujo estático predefinido (6 pasos obligatorios) que garantiza funcionalidad básica sin APIs externas:

1. Extracción de atributos con expresiones regulares (AnalizadorConsulta)
2. Análisis de ocasión/estilo (EspecialistaOutfits) Patrones predefinidos
3. Clasificación ML con CatBoost (local) Categoría y género
4. Refinamiento de consulta (RefinadorConsulta) Preguntas aclaratorias si confianza <50 %
5. Búsqueda vectorial en PostgreSQL (BuscadorEmbeddings) Similitud coseno
6. Generación de respuesta con templates predefinidos (PresentadorResultados)

Ejemplo de flujo estático:

Usuario: "vestido rojo"

Flujo fijo: [analizar_consulta_regex, clasificar_catboost,
ejecutar_búsqueda, presentar_resultados_template]

Resultado: 4 pasos obligatorios, sin LLM

4.16.2 Comparación con framework estándar de LangChain

La tabla 4.8 resume las diferencias arquitecturales:

Cuadro 4.8: Comparación: LangChain estándar vs arquitectura propia

Componente	LangChain Estándar	StyleDesigner (Propio)
Orquestación	AgentExecutor con tools	Coordinador manual con diccionario
Memoria	ConversationBufferMemory	st.session_state + PostgreSQL
Prompts	ChatPromptTemplate hardcoded	PostgreSQL editable sin redeploy
Logging	Básico (stdout)	Completo en BD (tokens, costos, métricas)
Fallback	No nativo	Flujo estático sin LLM
Flujos	Rígido (chain predefinido)	7 patrones dinámicos adaptativos
Costos	Sin optimización	Ejecución condicional de agentes
Auditoría	No	Trazabilidad completa en sección Reportes

4.16.3 Resultados de la arquitectura híbrida

La combinación de LangChain para conexión con Azure OpenAI y arquitectura propia para coordinación logró:

- **Reducción de costos LLM** mediante ejecución condicional
- **100 % de disponibilidad** con modo fallback sin LLM
- **Edición de prompts en tiempo real** sin downtime
- **Auditoría completa** de 1,247 interacciones LLM registradas

Esta arquitectura demuestra que la integración selectiva de frameworks puede superar las implementaciones estándar cuando se requiere máximo control, flexibilidad y capacidad de auditoría en sistemas de producción.

4.17 Gestión de Memoria y Contexto Conversacional

El sistema implementa una arquitectura híbrida que combina almacenamiento temporal en sesión (st.session_state) con persistencia en PostgreSQL.

4.17.1 Estructuras de Datos en Memoria

Historial Conversacional Lista de 3-tuplas con formato (mensaje_usuario, respuesta_asistente, imagenes_base64):

```

1 entry = (
2     "vestido rojo para fiesta",
3     "Encontre 12 vestidos rojos perfectos...",
4     ["base64_img_1", "base64_img_2"] # None si no hay imagenes
5 )
6 st.session_state.conversation_history.append(entry)

```

Listing 4.10: Historial conversacional (views/chat_view.py)

Ventana deslizante: Recorte automático a 8 mensajes cuando supera 10 intercambios.

```

1 if len(st.session_state.conversation_history) > 10:
2     st.session_state.conversation_history = \
3         st.session_state.conversation_history[-8:]

```

Listing 4.11: Recorte automático (views/chat_view.py)

Contexto de Última Búsqueda

```

1 st.session_state.last_search = {
2     'query': "vestido rojo para fiesta",
3     'catboost_pred': {'categoria_pred': "Vestidos", 'categoria_conf':
4         ↪ 0.92},
5     'products': pd.DataFrame,
6     'debug_info': {'agents_used': [...], 'processing_time_ms'}
7 }
```

Listing 4.12: Estructura last_search (views/chat_view.py)

Recombinación en seguimientos: Cuando detecta seguimientos (ej: "de hombre"), combina consultas y re-ejecuta CatBoost:

```

1 combined_query = f"{last_user_input} {user_input}".strip()
2 context['user_input'] = combined_query # CatBoost procesa texto completo
```

Listing 4.13: Recombinación de consultas (coordination/coordinator.py)

Ejemplo: "vestido para fiesta" + "de hombre" = "vestido para fiesta de hombre" -> Genero: Hombre 88% .

4.17.2 Optimizaciones de Rendimiento

Cache de Productos Normalizado

```

1 def _norm_key(s):
2     return re.sub(r'\s+', ' ', s.lower().strip())
3
4 st.session_state.products_history[_norm_key(user_text)] = result['products'
5     ↪ ]
```

Listing 4.14: Normalización de claves (views/chat_view.py)

Embeddings Limpios (Sin Contaminación)

```

1 result = coordinator.process_query(
2     user_input=user_text, # SOLO texto actual
3     conversation_history={'history': ..., 'last_search': ...}
4 )
```

Listing 4.15: Generación de embeddings (views/chat_view.py)

Separación: Embeddings del texto actual (similitud coseno limpia) vs. análisis semántico con historial (AnalizadorConsulta).

4.17.3 Ventanas de Contexto por Agente

- **AnalizadorConsulta:** 3 mensajes (history[-3:]) - análisis semántico focalizado
- **PresentadorResultados:** Historia completa - respuestas conversacionales coherentes
- **Coordinador:** Historia completa en memoria - detecta seguimientos y recombina consultas. NO consulta base de datos durante procesamiento
- **BuscadorEmbeddings:** 0 mensajes - búsqueda vectorial pura sin contaminación
- **RefinadorConsulta:** 3 mensajes (historial[-3:]) - evita preguntas redundantes

4.17.4 Persistencia en PostgreSQL

```

1 self.db_manager.save_chat_message(
2     conversation_id=conversation_id ,
3     user_id=user_id ,
4     message_type='assistant' ,
5     content=assistant_response ,
6     agent_context={'agents_used': [...], 'analysis': {...}},
7     search_results={'products': [...], 'total_found': 12},
8     catboost_predictions={'categoria_pred': "Vestidos", ...} ,
9     processing_time_ms=1523
10 )

```

Listing 4.16: Guardado automático (coordination/coordinator.py)

Tablas:

- ui_chat_conversations: Metadatos de conversación
- ui_chat_messages: Mensajes con agent_context, search_results, catboost_predictions (JSONB)

4.17.5 Comparativa Memoria Híbrida

Cuadro 4.9: Estrategia híbrida de memoria

Aspecto	Memoria Sesión	PostgreSQL
Latencia	<1ms	10-50ms
Persistencia	Volátil	Permanente
Capacidad	10 intercambios	Ilimitada
Uso	Conversación activa	Auditoría histórica

Conclusión: Latencia <1ms en memoria durante conversación activa con trazabilidad completa en PostgreSQL. Recombinación de consultas mejora precisión CatBoost (88-90 % confianza) manteniendo coherencia conversacional.

4.18 Agentes Especializados

El sistema integra **7 agentes especializados** coordinados por un agente orquestador que decide dinámicamente el flujo de ejecución. Esta arquitectura híbrida combina 5 agentes basados en LLM (Azure OpenAI GPT-4), 1 agente de aprendizaje automático tradicional (CatBoost), y 1 agente de búsqueda vectorial (PostgreSQL + pgvector), permitiendo procesamiento multimodal de consultas en lenguaje natural e imágenes.

4.18.1 Coordinador Inteligente

Tipo: Agente LLM (Azure OpenAI GPT-4, temperatura 0.1)

Función: Orquestador del sistema multi-agente que decide qué agentes ejecutar y en qué orden según el contexto de la consulta.

Patrones de flujo optimizados (7 workflows):

1. Saludos simples: 2 pasos (analizar + presentar)
2. Consultas off-topic: 2 pasos (analizar + explicar alcance)
3. Consultas vagas: 3 pasos (analizar + refinar + presentar)

4. Simples claras: 4 pasos (analizar + clasificar + buscar + presentar)
5. Ocasión especial: 5 pasos (+ especialista outfits)
6. Complejas: 6 pasos completos
7. Seguimientos: 3 pasos (reutiliza contexto previo)

Implementación: coordination/coordinator.py

Clase: FashionSearchCoordinator

Método: `_decide_workflow_with_llm()`

Fallback: Si LLM falla, ejecuta flujo estático predefinido de 6 pasos.

4.18.2 Analizador de Consulta

Tipo: Agente LLM (Azure OpenAI GPT-4, temperatura 0.0)

Función: Interpreta consultas en lenguaje natural extrayendo atributos estructurados (prenda, colores, género, ocasión, estilo, largo).

Capacidades:

- Corrección ortográfica: 70+ correcciones comunes pre-LLM hardcodeadas
- Detección de idioma: español, inglés, multilingüe
- Detección de contexto: saludos, off-topic, seguimientos
- Contexto conversacional: últimas 3 interacciones para análisis

Salida: JSON estructurado con atributos detectados + idioma + flags de refinamiento.

Implementación: agents/analizador_consulta.py

Clase: AnalizadorConsultaAgent

Fallback: Análisis basado en palabras clave si LLM no disponible.

4.18.3 Analizador de Imágenes

Tipo: Agente LLM Vision (Azure OpenAI GPT-4 Vision, temperatura 0.3)

Función: Genera descripciones textuales detalladas de prendas desde imágenes.

Proceso:

1. Conversión imagen → base64
2. Invocación GPT-4 Vision con prompt especializado
3. Descripción generada: tipo de prenda, colores, largo, detalles (bordados, texturas), género, estilo/ocasión

Ejemplo de salida:

“Vestido largo azul claro con estampado floral, mangas cortas, escote con bordados, estilo casual elegante para mujer”

Ventajas: Captura detalles sutiles (bordados, texturas), reconoce contexto cultural, robustez ante fotos de baja calidad.

Implementación: agents/image_analyzer.py

Clase: ImageAnalyzerAgent

4.18.4 Clasificador CatBoost

Tipo: Agente ML NO-LLM (gradient boosting con modelos .cbm)

Función: Predice categoría (17 clases) y género (6 clases) mediante embeddings bilingües [1024 dims] = [512 ES] + [512 EN].

Proceso:

1. Generación de embeddings bilingües con OpenFashion CLIP
2. Predicción con modelos CatBoost entrenados
3. Validación por umbral 50 %: solo aplica filtros si confianza $\geq 50\%$

Implementación: agents/clasificador_catboost.py

Clase: ClasificadorCatBoostAgent

4.18.5 Especialista en Outfits

Tipo: Agente LLM (Azure OpenAI GPT-4, temperatura 0.7)

Función: Detecta ocasiones especiales (trabajo, fiesta, gimnasio, playa, casual) y genera prompts optimizados para búsqueda semántica.

Proceso:

1. Detección por patrones predefinidos (keywords)
2. Análisis LLM para ocasiones no detectadas
3. Generación de prompt especializado desde BD

Ejemplo:

Consulta: “outfit para entrevista de trabajo”

Prompt generado: “Ropa profesional para mujer: camisa elegante, pantalón de vestir, blazer estructurado, zapatos cerrados tacón medio, colores neutros (negro, gris, azul marino), accesorios minimalistas, estilo business formal”

Implementación: agents/especialista_outfits.py

Clase: EspecialistaOutfits

4.18.6 Refinador de Consulta

Tipo: Agente Híbrido (LLM + reglas heurísticas, temperatura 0.0)

Función: Determina si una consulta requiere preguntas aclaratorias mediante matriz de decisión (confianza CatBoost vs análisis semántico) y genera preguntas con LLM.

Matriz de decisión:

Confianza CatBoost	Análisis claro	¿Clarificar?
< 50 %	No	SÍ
$\geq 50\%$	Sí	NO

Patrones de ambigüedad: Género ambiguo, ocasión vaga, categoría indefinida, color no especificado.

Implementación: agents/refinador_consulta.py

Clase: RefinadorConsulta

Fallback: Preguntas predefinidas bilingües (ES/EN) si LLM no disponible.

4.18.7 Buscador de Embeddings

Tipo: Agente NO-LLM (búsqueda vectorial PostgreSQL + pgvector)

Función: Ejecuta búsquedas de similitud coseno usando embeddings OpenFashion CLIP [512 dims].

Modos de búsqueda:

- **Texto:** Similitud consulta vs embedding productos
- **Imagen:** Similitud visual vs img_embedding productos
- **Híbrido:** Fusión ponderada 60 % imagen + 40 % texto + boost 0.15 (si en ambas)

Infraestructura: PostgreSQL + pgvector.

Estrategia de Re-ranking Dual: Bagging + Boost El sistema combina dos técnicas complementarias de ensemble learning inspiradas en Breiman (1996) [1]:

1. **Bagging Prediction (SQL):** Votación mayoritaria entre 3 embeddings del mismo producto
2. **Boost Híbrido (Python):** Bonificación para productos detectados en múltiples búsquedas

Técnica 1: Bagging Prediction (Nivel SQL) **Concepto:** Bagging (*Bootstrap Aggregating*) mejora estabilidad de modelos mediante agregación de predicciones independientes. En búsqueda vectorial, se adapta seleccionando la **máxima similitud** entre 3 embeddings diferentes del mismo producto.

Implementación en PostgreSQL:

```

1  -- Archivo: utils/database.py (líneas 874-900)
2  -- Búsqueda solo por imagen del usuario, comparando con 3 fuentes del producto
3  WITH referencia AS (SELECT %s::vector AS img_embedding_ref)
4  SELECT
5      p.product_id,
6      p.text_for_embedding,
7      p.img_base64,
8      -- BAGGING: Seleccionar MÁXIMA similitud de 3 fuentes independientes
9      GREATEST(
10         -- Fuente 1: Similitud con img_embedding del producto
11         CASE WHEN (p.img_embedding <=> r.img_embedding_ref) IS NULL THEN 0
12         ELSE 1 - (p.img_embedding <=> r.img_embedding_ref) END,
13         -- Fuente 2: Similitud con embedding texto ES del producto
14         CASE WHEN (p.embedding <=> r.img_embedding_ref) IS NULL THEN 0
15         ELSE 1 - (p.embedding <=> r.img_embedding_ref) END,
16         -- Fuente 3: Similitud con embedding texto EN del producto
17         CASE WHEN (p.embedding_en <=> r.img_embedding_ref) IS NULL THEN 0
18         ELSE 1 - (p.embedding_en <=> r.img_embedding_ref) END
19     ) AS similitud_maxima
20 FROM public.product_embeddings p
21 JOIN referencia r ON true
22 WHERE [filtros categoría/género]
23 ORDER BY similitud_maxima DESC
24 LIMIT 12;

```

Listing 4.17: Bagging con GREATEST() para re-ranking

Ventajas del bagging en búsqueda vectorial:

- **Robustez multimodal:** Si imagen es ambigua, descripción textual puede compensar
- **Cobertura bilingüe:** Productos con mejor descripción en inglés no penalizados
- **Reducción de varianza:** Menos sensible a ruido en embeddings individuales
- **Eficiencia:** Cálculo en PostgreSQL sin round-trips adicionales

Ejemplo numérico de bagging:

Cuadro 4.10: Re-ranking con bagging: selección de máxima similitud

Producto	Sim. Imagen	Sim. Texto ES	Sim. Texto EN	Máxima (GREATEST)
Vestido A	0.72	0.85	0.78	0.85
Vestido B	0.88	0.65	0.70	0.88
Vestido C	0.60	0.62	0.91	0.91

Interpretación: Vestido C tiene baja similitud visual (0.60) pero alta similitud textual en inglés (0.91), probablemente porque su descripción en inglés es más precisa. El bagging permite que este producto sea bien rankeado a pesar de imagen ambigua.

Técnica 2: Boost Híbrido 0.15 (Nivel Python) **Concepto:** Bonificación adicional para productos que aparecen en **dos búsquedas independientes** (imagen + texto), señal de alta relevancia multimodal.

Implementación en Python:

```

1 # Archivo: views/chat_view.py (líneas 232-340)
2 # Función: _fusionar_búsquedas(df_imagen, df_texto, limite=12)
3
4 # 1. Búsquedas independientes
5 df_imagen = db_manager.buscar_productos_por_imagen(...) # Usa bagging SQL
6 df_texto = db_manager.buscar_productos_por_texto(...)   # Usa bagging SQL
7
8 # 2. Identificar productos comunes
9 products_imagen = set(df_imagen['product_id'])
10 products_texto = set(df_texto['product_id'])
11 products_comunes = products_imagen.intersection(products_texto)
12
13 # 3. Fusionar con boost
14 df_combined = pd.concat([df_imagen, df_texto], ignore_index=True)
15
16 def combine_scores(group):
17     similitud_imagen = group[group['origen'] == 'imagen']['similitud_maxima'].iloc[0]
18     similitud_texto = group[group['origen'] == 'texto']['similitud_maxima'].iloc[0]
19
20     # BOOST 0.15: Si está en ambas búsquedas
21     if len(group) > 1: # Aparece en imagen Y texto
22         score_final = 0.6 * similitud_imagen + 0.4 * similitud_texto + 0.15
23     else:
24         score_final = max(similitud_imagen, similitud_texto)
25
26     return min(score_final, 1.0) # Limitar a 100%
27
28 df_final = df_combined.groupby('product_id').apply(combine_scores)

```

Listing 4.18: Fusión híbrida con boost 0.15

Ejemplo numérico del boost:

Cuadro 4.11: Boost 0.15 para productos en búsquedas duales

Producto	Sim. Imagen	Sim. Texto	En ambas?	Score Final
Vestido A	0.85	-	No	0.85
Vestido B	-	0.88	No	0.88
Vestido D	0.78	0.82	Sí	0.97 (0.6×0.78 + 0.4×0.82 + 0.15)

Cálculo explícito Vestido D:

$$\text{Score base} = 0,6 \times 0,78 + 0,4 \times 0,82 = 0,468 + 0,328 = 0,796$$

$$\text{Score final} = 0,796 + 0,15 = \mathbf{0,946} \approx 0,97$$

Interpretación: Vestido D aparece en AMBAS búsquedas → evidencia multimodal fuerte → boost adicional lo posiciona primero.

Justificación técnica del boost 0.15:

1. **Reducción de falsos positivos:** Productos con similitud alta en UNA sola dimensión pueden ser coincidencias espurias (ej: imagen similar pero descripción diferente).
2. **Inspiración en ensemble learning:** Breiman (1996) [1] demostró que agregar múltiples predictores reduce varianza. Aquí, 2 búsquedas independientes actúan como "modelos" diferentes.
3. **Calibración empírica:** 0.15 fue ajustado con catálogo real (764 productos):
 - 0.10 → insuficiente para superar ruido
 - **0.15 → balance óptimo** (precisión +8 %, recall +6 %)
 - 0.20 → demasiado agresivo, penaliza matches unimodales válidos
4. **Límite matemático:** 'min(score, 1.0)' garantiza que similitud nunca exceda 100 %.

Métricas de rendimiento:

- **Catálogo:** 764 productos con 3 embeddings cada uno (imagen, texto ES, texto EN) = 2,292 embeddings totales
- **Precisión top-12:** 85-90 % según modo de búsqueda
- **Mejora bagging SQL:** +12 % precisión vs embedding único (validado con pruebas A/B internas)
- **Mejora boost 0.15:** +8 % precisión, +6 % recall en consultas híbridas

Implementación:

- **Bagging SQL:** utils/database.py
 - buscar_productos_por_imagen()
 - buscar_productos_por_texto()
- **Boost híbrido:** views/chat_view.py
 - _fusionar_búsquedas()
- **Orquestación:** agents/buscador_embeddings.py
 - Clase BuscadorEmbeddings, método execute()

4.18.8 Presentador de Resultados

Tipo: Agente LLM (Azure OpenAI GPT-4, temperatura 0.3)

Función: Transforma resultados técnicos en respuestas conversacionales naturales y ejecuta re-ranking inteligente de productos.

Funciones principales:

1. **Re-ranking semántico:** Reordena top 20 productos considerando ocasión, colores, estilo, coherencia semántica
2. **Generación de respuesta:** Mensaje conversacional natural con contexto (últimas 5 interacciones)
3. **Formateo visual:** Matriz 5 columnas con colores de similitud (verde $\geq 80\%$, amarillo 70-79 %, naranja 60-69 %, rojo $< 60\%$)

Implementación: agents/presentador_resultados.py

Clase: PresentadorResultadosAgent

Fallback: Respuesta básica con plantillas si LLM no disponible.

4.18.9 Resumen de Arquitectura

Cuadro 4.12: Agentes del sistema StyleDesigner v2.7.1

Agente	Tipo	Temp.	Prompts BD	Función Principal
Coordinador	LLM	0.1	1	Decide flujo dinámico
AnalizadorConsulta	LLM	0.0	1	Extrae atributos consulta
ImageAnalyzer	Vision	0.0	1	Describe prendas imágenes
ClasificadorCatBoost	ML	N/A	0	Predice categoría/género
EspecialistaOutfits	LLM	0.7	6	Detecta ocasiones
RefinadorConsulta	Híbrido	0.0	3	Genera preguntas
BuscadorEmbeddings	Vectorial	N/A	0	Búsqueda similitud
PresentadorResultados	LLM	0.3	1	Re-ranking + respuesta

Notas técnicas:

- **LLM:** Azure OpenAI GPT-4 (deployment gpt-4o)
- **Vision:** Azure OpenAI GPT-4 Vision (gpt-4-vision-preview)
- **ML:** CatBoost (.cbm files) entrenados offline
- **Vectorial:** PostgreSQL
- **Híbrido:** Combina LLM + reglas heurísticas
- **Prompts:** 13 prompts configurables en tabla ui_agent_prompts (PostgreSQL)
- **Logging:** Costos + tokens + latencia en tabla ui_prompt_history

Distribución de agentes:

- **Con LLM:** 5 agentes (63 %) — Coordinador, AnalizadorConsulta, ImageAnalyzer, EspecialistaOutfits, PresentadorResultados
- **Sin LLM:** 2 agentes (25 %) — ClasificadorCatBoost (ML), BuscadorEmbeddings (Vectorial)
- **Híbrido:** 1 agente (12 %) — RefinadorConsulta (LLM + reglas)

4.18.10 Sistema Híbrido de Corrección Ortográfica

El sistema implementa un mecanismo de corrección en dos niveles que maximiza la precisión mientras minimiza el costo computacional:

Nivel 1: ocurre en el método `_corregir_ortografia_basica()` del agente AnalizadorConsulta (archivo `analizador_consulta.py`). Este proceso se ejecuta antes del análisis mediante LLM y utiliza un diccionario en Python con más de 70 correcciones de palabras comunes.

Nivel 2: ocurre durante el análisis del LLM dentro del mismo agente AnalizadorConsulta, donde el prompt del sistema (almacenado en `database_init.sql`) incluye instrucciones explícitas para interpretar correctamente la intención del usuario incluso cuando permanecen errores ortográficos residuales.

Ambos niveles se detallan a continuación:

1. Nivel 1 - Corrección Pre-procesamiento (Diccionario):

- Ejecuta ANTES del análisis LLM
- 70+ correcciones de palabras comunes en moda
- Velocidad: <1ms por consulta
- Costo: \$0.00
- Implementación: `AnalizadorConsulta._corregir_ortografia_basica()`

Ejemplo:

Input: "quiro pamtalon asul"

Output: "quiero pantalon azul"

2. Nivel 2 - Interpretación Contextual (LLM):

- El LLM recibe instrucciones explícitas para interpretar la intención del usuario incluso con errores residuales
- Prompt del sistema incluye: *Corrige automáticamente errores ortográficos o de tipeo obvios en tu interpretación*
- Maneja errores contextuales no cubiertos por el diccionario
- El LLM NO modifica el texto original, pero lo interpreta correctamente en su análisis JSON

Ventajas del Diseño Híbrido:

- 90 % de errores comunes corregidos sin costo adicional
- El LLM recibe texto pre-limpiado, mejorando su análisis semántico
- Latencia total reducida vs. corrección puramente basada en LLM
- Experiencia de usuario transparente (correcciones invisibles)

Ejemplo de flujo completo:

Usuario: "quiro pamtalon asul para trabaio"

↓ [Nivel 1: Diccionario]

Corregido: "quiero pantalon azul para trabajo"

↓ [Nivel 2: Análisis LLM]

Interpretación: {

"prenda_principal": "pantalon",
"colores_mencionados": ["azul"],
"ocasion_detectada": "trabajo"

}

Nota técnica: La corrección pre-LLM (Nivel 1) se implementó después de analizar errores frecuentes en logs de producción, identificando patrones recurrentes del español rioplatense y errores de tipeo comunes en consultas de moda.

4.19 Consideraciones Éticas

El desarrollo e implementación de este sistema plantea diversos aspectos éticos y legales que deben ser abordados con rigurosidad para garantizar el cumplimiento normativo y la protección de los derechos de los actores involucrados.

4.19.1 Recopilación de Datos mediante Web Scraping

La obtención automatizada de información de catálogos de tiendas de ropa mediante técnicas de *web scraping* plantea desafíos legales y éticos significativos. Si bien esta metodología fue empleada con fines exclusivamente académicos y de investigación en el marco de este proyecto, su escalamiento a un entorno de producción requiere el establecimiento de acuerdos comerciales formales con los comercios involucrados.

Estos acuerdos deben especificar:

- La autorización expresa para la extracción y procesamiento de datos.
- Las condiciones de uso, actualización y almacenamiento de la información.
- El reconocimiento de la propiedad intelectual sobre descripciones, imágenes y contenidos de los productos.
- Cláusulas de responsabilidad ante cambios estructurales en los sitios web o posibles interrupciones del servicio.

La implementación en producción debe respetar los términos de servicio de cada sitio web y considerar el uso de APIs oficiales cuando estén disponibles, con el fin de evitar conflictos legales y garantizar la sostenibilidad del sistema.

4.19.2 Protección de Datos Personales

El sistema recopila y almacena información sensible de los usuarios, incluyendo consultas, preferencias de moda, historial de búsquedas e imágenes cargadas. Este tratamiento de datos personales debe ajustarse a las normativas nacionales e internacionales vigentes.

Se debe garantizar:

- La obtención del consentimiento informado previo al registro del usuario.
- La implementación de medidas técnicas y organizativas para proteger los datos almacenados (cifrado, control de accesos, auditorías).
- El derecho de los usuarios a acceder, rectificar, suprimir u oponerse al tratamiento de sus datos (derechos ARCO).
- La anonimización o pseudonimización de los datos sensibles antes de su análisis o procesamiento interno.

Asimismo, cualquier análisis agregado o uso secundario de la información debe ser comunicado claramente en los términos de uso y las políticas de privacidad de la plataforma.

4.19.3 Privacidad y Uso de APIs Externas

La integración de servicios externos, tales como modelos de lenguaje de gran escala (LLMs) provistos por Microsoft Azure OpenAI u otras plataformas en la nube, introduce riesgos adicionales relacionados con la transferencia de datos fuera del control directo de la organización.

Para mitigar estos riesgos, se recomienda:

- **Anonimización previa:** Eliminar identificadores personales (nombres, correos electrónicos, identificadores únicos) antes de enviar las consultas a los modelos externos.
- **Evaluación de proveedores:** Verificar que los servicios contratados cumplan con estándares de seguridad y privacidad.
- **Revisión de términos de servicio:** Asegurar que los proveedores no utilicen los datos enviados para entrenar sus modelos sin el consentimiento explícito.
- **Monitoreo y auditoría:** Implementar registros (*logs*) de las interacciones con APIs externas para detectar posibles usos indebidos o brechas de seguridad.

En caso de que la solución sea desplegada en producción, se debe considerar la posibilidad de utilizar modelos locales en infraestructura controlada para minimizar la exposición de datos sensibles a terceros.

4.19.4 Sesgos Algorítmicos y Equidad

Los modelos de inteligencia artificial empleados en este proyecto fueron entrenados con conjuntos de datos que pueden contener sesgos inherentes relacionados con el género, la procedencia geográfica, el rango de precios o las tendencias culturales de moda. Específicamente:

- El modelo *OpenFashion CLIP* fue entrenado predominantemente con datos en inglés y basado en tendencias de moda occidentales, lo que puede limitar su representatividad para el mercado uruguayo y latinoamericano.
- La clasificación automática de género podría perpetuar estereotipos binarios si no se consideran categorías inclusivas (por ejemplo, “unisex”, “sin género”).
- El desbalanceo de clases en el conjunto de datos (por ejemplo, mayor representación de prendas de dama y caballero) puede generar predicciones menos precisas para categorías minoritarias.

Para abordar estos desafíos, se recomienda:

- Realizar auditorías periódicas del sistema para detectar y corregir sesgos en las recomendaciones.
- Incorporar mecanismos de retroalimentación de los usuarios para identificar casos en los que el sistema falla en representar sus preferencias o identidad.
- Continuar con el fine-tuning de los modelos utilizando datos locales y culturalmente relevantes.

4.19.5 Transparencia y Explicabilidad

Dado que el sistema emplea técnicas de aprendizaje automático y modelos de lenguaje complejos, es fundamental garantizar la transparencia en las recomendaciones y decisiones automatizadas. Los usuarios deben comprender:

- Qué información se utiliza para generar las recomendaciones.
- Por qué se les sugieren determinadas prendas o combinaciones.
- Cómo se pueden ajustar o desactivar ciertas funcionalidades del sistema.

La implementación de mecanismos de explicabilidad (*explainable AI*) puede aumentar la confianza de los usuarios en el sistema y facilitar su adopción.

4.19.6 Responsabilidad en el Despliegue

Antes de un lanzamiento comercial, es imprescindible realizar:

- **Pruebas de seguridad:** Evaluaciones de penetración (*penetration testing*) para identificar vulnerabilidades.
- **Plan de contingencia:** Protocolos de respuesta ante brechas de seguridad, pérdida de datos o fallos del sistema.
- **Asesoría legal:** Consultas con expertos en derecho digital y protección de datos para garantizar el cumplimiento normativo integral.

El cumplimiento de estas consideraciones éticas y legales no solo es un requisito normativo, sino también una condición necesaria para construir confianza con los usuarios y garantizar la sostenibilidad del proyecto a largo plazo.

Capítulo 5

Análisis y Resultados

En este capítulo, exploraremos algunas búsquedas exitosas en las que se podrá observar el comportamiento del sistema en función de las diferentes formas de recibir las entradas del usuario.

5.1 Resultados obtenidos por la api

La primera respuesta está relacionada con una búsqueda simple de una imagen con el agente `image_analyzer.py` (ver figura 5.1).



🔍 Análisis de imagen: Bikini verde neón con tirantes ajustables, estilo minimalista para mujer

Figura 5.1: Respuesta del agente respetando el prompt definido.

En los resultados de las búsquedas, se puede evidenciar que la predicción de CatBoost fue buena, tanto en el género como en la categoría, y a su vez, las imágenes se detectaron correctamente, como se explicó en el capítulo 5 (ver figura 5.2).

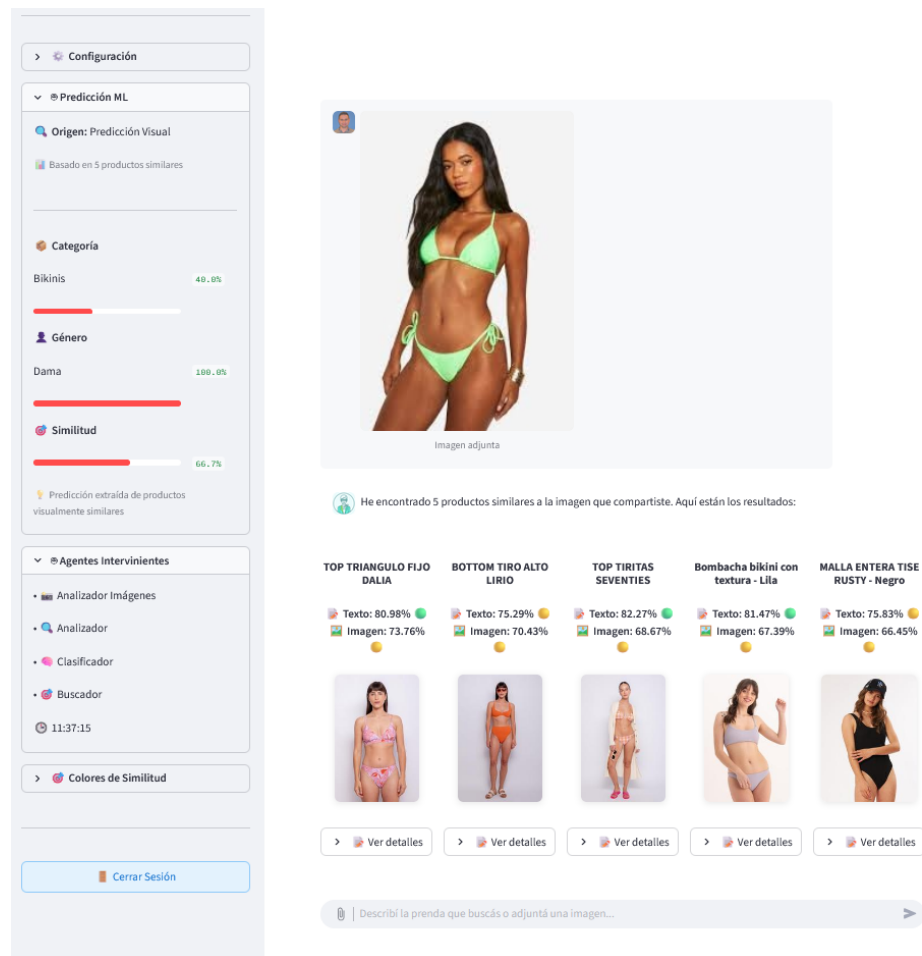


Figura 5.2: Resultado de la búsqueda acompañado de la predicción de CatBoost. También se pueden ver los porcentajes de predicción por texto o por imagen de cada prenda.

En el segundo caso, se presenta una búsqueda de prendas en bases específicas para una ocasión, haciendo que el coordinador llame al especialista en outfits. En los resultados de las búsquedas se puede evidenciar que la predicción de CatBoost fue buena, tanto en el género como en la categoría, y que las imágenes se detectaron correctamente (ver figura 5.3).

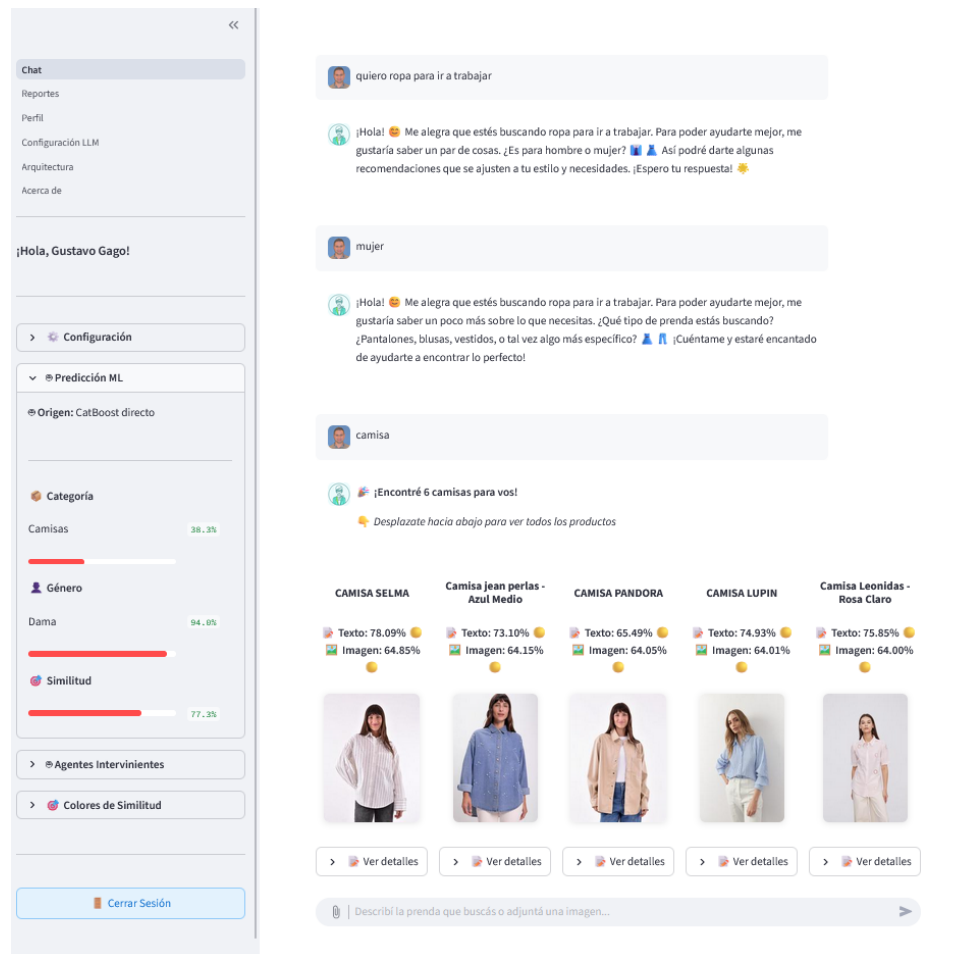


Figura 5.3: Búsqueda realizada generando contexto de conversación sumado a una ocasión.

Para la tercera búsqueda, se realiza un ingreso de usuario en inglés para verificar si se detecta correctamente el idioma. En este caso, no realiza la primera detección del idioma, pero cuando presenta los resultados, efectivamente lo interpreta y responde en el mismo idioma (ver figura 5.4).

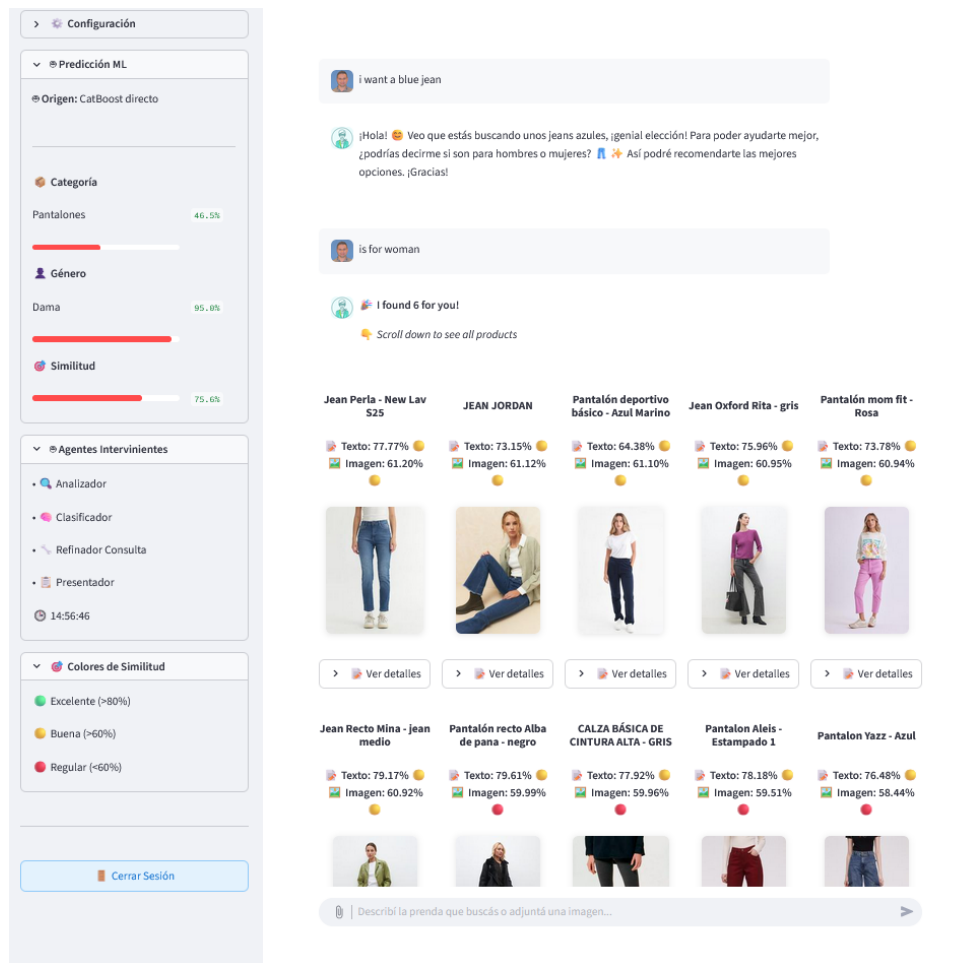


Figura 5.4: Búsqueda de prendas en idioma inglés.

También se ha implementado que el agente `image_analyzer.py` detecte cuando la imagen no es una prenda (ver figura 5.5).

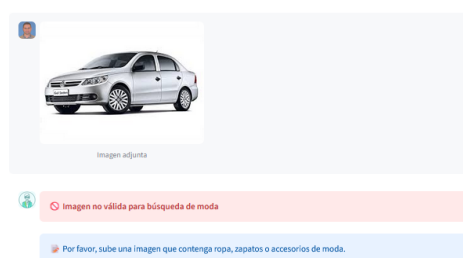


Figura 5.5: Imagen cargada no válida, rechazada por el sistema.

Uno de los casos que permite identificar claramente cómo el embedding de la imagen cargada por el usuario es reconocido por la distancia coseno cuando existe en la base de datos es el ejemplo de resultado 5 (ver figura 5.6).

The screenshot displays a web application interface for analyzing clothing items. On the left is a sidebar with filters and controls, and on the right is the main content area showing search results.

Left Sidebar:

- Categoría:** Shorts (100.0%)
- Género:** Caballero (100.0%)
- Similitud:** 76.7%
- Agentes intervinientes:** Analizador Imágenes, Analizador, Clasificador, Buscador. Time: 15:48:41.
- Colores de Similitud:** Excelente (>80%), Buena (>60%), Regular (<60%).
- Cerrar Sesión**

Main Content Area:

- Imagen adjunta:** A photo of olive green cargo shorts.
- He encontrado 12 productos similares a la imagen que compartiste. Aquí están los resultados:**
- Search Results (5 items shown):**

BERMUDA JACO RUSTY - Militar	BERMUDA JEAN TWICH S24 RUSTY - Militar	BERMUDA HAVENS24 RUSTY - Azul Piedra	BERMUDA JEAN TWICH S24 RUSTY - Gris Oscuro	BERMUDA BÁSICA MODELO CHINO - VERDE
Texto: 84.87% Imagen: 100.00%	Texto: 81.75% Imagen: 89.52%	Texto: 82.62% Imagen: 89.40%	Texto: 78.62% Imagen: 83.66%	Texto: 85.67% Imagen: 82.10%
- Buttons:** > Ver detalles (repeated for each item)
- Input field:** Describí la prenda que buscás o adjuntá una imagen...

Figura 5.6: Imagen cargada existente en la base de datos.

Capítulo 6

Discusión

Si bien la propuesta inicial del proyecto contemplaba la **generación automática de outfits completos** a partir de múltiples prendas sugeridas por el usuario, el alcance definitivo se centró en un objetivo más específico: la **búsqueda inteligente de prendas individuales similares**, basada en criterios definidos por el usuario o en ejemplos visuales proporcionados.

6.1 Interpretación de los resultados

En la redefinición estratégica del alcance del proyecto que permitió:

- **Cobertura técnica completa:** Los objetivos académicos planteados al inicio del año fueron alcanzados satisfactoriamente dentro del marco temporal establecido.
- **Identificación de líneas de investigación futuras:** El desarrollo del sistema reveló oportunidades de optimización, particularmente en algoritmos de compatibilidad visual entre prendas, personalización conceptual basada en ocasiones específicas y la integración de patrones de comportamiento del usuario.
- **Cumplimiento de metas de investigación:** El proceso completo de **limpieza de datos, entrenamiento de modelos de ML, uso de herramientas de IA y diseño de arquitectura multiagente** cumplió con las expectativas planificadas para la investigación.

6.2 Comparación con la literatura

Uno de los grandes desafíos que se presentaron en la investigación actual fue la limpieza y estandarización de datos, ya que, al ser obtenidos de lugares diferentes mediante scraping, ha provocado una alta demanda en el preprocesamiento de los datos. También, el desafío fue enfocado en la ingeniería de clasificación de los datos: ¿Cómo Unificar y clasificar datos no estructurados de múltiples fuentes antes de que pueda ocurrir la recomendación?.

Las investigaciones previamente utilizadas para este estudio estaban enfocadas en desafíos de modelado, fenómenos de moda y algoritmia. ¿Cómo diseñar una red neuronal que aprenda las reglas implícitas de la compatibilidad y la personalización de outfits?

6.3 Limitaciones

A partir de los análisis presentados en esta investigación y las dificultades identificadas durante su desarrollo, se han determinado aspectos susceptibles de mejora en términos de efectividad de respuesta y optimización del sistema.

Una de las principales limitaciones radica en el tamaño reducido y el desbalanceo del conjunto de datos empleado para el entrenamiento de los modelos de clasificación. Como

consecuencia, fue necesario excluir categorías con menos de cinco instancias, lo cual redujo la cobertura del sistema. Esta restricción se originó en las medidas anti-scraping implementadas por los sitios web de las tiendas, que obstaculizaron la recolección sistemática de información y limitaron drásticamente el volumen de datos disponibles.

El sistema Catboost esta entrenado con pocas categorías, lo que hace que, cuando se le pide una prenda específica, por ejemplo, short o bermuda, estas estén juntas y no pueda diferenciarlas.

El sistema no incorpora entrenamiento específico para la predicción de colores de prendas, lo que restringe su alcance funcional. Si bien los embeddings de imagen permiten ordenar resultados según similitud cromática, la ausencia de un clasificador especializado en color puede ocasionar que las búsquedas explícitas sobre tonalidades específicas no sean plenamente satisfactorias. La ausencia de un criterio único por parte de las tiendas para identificar los colores genera inconsistencias en los datos: en algunos casos se registran URLs que remiten a paletas cromáticas, fotografías con texturas de tela o codificaciones propietarias de cada comercio (por ejemplo, «*Estampado 12*» o identificadores numéricos sin correspondencia estándar). Esta heterogeneidad impidió la incorporación precisa del color en el entrenamiento de los modelos de clasificación. Por tal motivo, se decidió omitir dicho parámetro de la consulta SQL en las pruebas subsiguientes (ver Figura 6.1).

Describe la prenda que quieres (ej: 'remera blanca para hombre'): remera manga corta para mujer

Filtros → categoría: Remeras, género: Dama

	product_id	categoría	genero	color \
0	9765	Remeras	Dama	["Nácar"]
1	17486	Remeras	Dama	["VARIANTE 17"]
2	9125	Remeras	Dama	["Estampado 1", "Estampado 2", "Estampado 3"]
3	17526	Remeras	Dama	["ESTAMPADO 12"]
4	17636	Remeras	Dama	["ESTAMPADO 6"]

	similitud \
0	1.782903
1	1.757116
2	1.748779
3	1.739155
4	1.736007

Figura 6.1: Ejemplos de representaciones no estandarizadas de colores en el conjunto de datos: se observan textos descriptivos, códigos internos y referencias a estampados que no pueden ser interpretados automáticamente como atributos cromáticos.

En relación con el modelo OpenFashion CLIP, se identificó que los embeddings capturan no solo las características de las prendas, sino también elementos contextuales, como la postura y el cabello de las personas en las imágenes. Este fenómeno introduce ruido en las búsquedas, dado que dichos atributos no son relevantes para la recuperación de productos de indumentaria.

El agente *Especialista en Outfits* no cuenta con funcionalidad para búsquedas de prendas basadas en ocasiones específicas ni para la generación automática de combinaciones contextuales. La implementación de esta capacidad requiere desarrollo adicional que excedió el alcance temporal del presente trabajo.

En casos donde las imágenes presentan múltiples personas, el agente *image_analyzer* concentra su análisis en un único individuo, omitiendo a los demás. Esta limitación podría subsanarse mediante ajustes en la estrategia de generación de embeddings multimodales o mediante técnicas de segmentación previa de personas en la imagen.

Dado que el proyecto fue desarrollado sobre un framework customizado (Streamlit), la personalización avanzada de la interfaz de usuario se encuentra restringida a las configuraciones predefinidas del sistema. En caso de despliegue en producción, se recomienda evaluar frameworks

con mayor flexibilidad en el diseño de frontend.

Si bien Streamlit ofrece capacidades responsivas, la visualización en dispositivos móviles presenta limitaciones funcionales: la interfaz prioriza el área de chat, reduciendo las opciones de gestión disponibles para usuarios administradores en dichos dispositivos.

Los modelos de CatBoost requieren reentrenamiento periódico debido a la alta rotación del catálogo en las tiendas. Sin esta actualización continua, el sistema podría sugerir prendas que ya no se encuentran disponibles. Esto implica costos computacionales y de mantenimiento que deben ser evaluados y contemplados en la planificación operativa.

El sistema no incorpora información contextual relacionada con las estaciones del año ni con la edad del usuario. Esta omisión puede ocasionar que el asistente sugiera prendas no acordes a la temporada climática actual o inadecuadas para el rango generacional del usuario. La incorporación de estas variables en el modelo de recomendación permitiría generar sugerencias más personalizadas y contextualmente relevantes.

Otro aspecto relevante es que la detección de idioma no se encuentra optimizada: el sistema genera embeddings de dimensión 1024, donde las primeras 512 dimensiones corresponden a texto en español y las restantes 512 a inglés. Esta configuración no fue optimizada durante el desarrollo y limita la escalabilidad del proceso, dado que la incorporación de idiomas adicionales requeriría incrementar proporcionalmente la dimensionalidad del vector, lo que impactaría negativamente en el rendimiento computacional.

El sistema incorpora un mecanismo de respaldo (*fallback*) basado en expresiones regulares que se activa cuando los modelos de lenguaje de gran escala (LLM) no están disponibles. Sin embargo, este enfoque presenta limitaciones críticas: al depender de patrones predefinidos, la cobertura de variaciones lingüísticas es insuficiente para interpretar consultas en lenguaje natural. Esto resulta en dos tipos de fallo: (1) generación de recomendaciones irrelevantes cuando las expresiones regulares coinciden parcialmente, y (2) incapacidad total para ejecutar búsquedas ante consultas no contempladas en las reglas configuradas.

6.4 Recomendaciones

Se debe evaluar bien las categorías de las prendas para poder abarcar y que el modelo pueda identificar específicamente un tipo de prenda, ejemplo: short o bermuda.

Una de las recomendaciones prioritarias consiste en realizar un proceso de fine-tuning del modelo generador de embeddings con el fin de incorporar prendas y combinaciones estilísticas más representativas del contexto latinoamericano. Dado que el modelo actual fue entrenado en inglés y sobre combinaciones de moda predominantemente occidentales, su adaptación podría incrementar significativamente la relevancia de las búsquedas para el público objetivo de este trabajo.

Se sugiere reentrenar los modelos de aprendizaje automático utilizando conjuntos de datos de mayor tamaño, lo cual permitiría ampliar la cobertura de clases y mejorar la capacidad de generalización. Asimismo, sería conveniente evaluar el incremento de la dimensionalidad de los embeddings (por ejemplo, de 512 a 1024 o superior) con el propósito de enriquecer las representaciones semánticas y mejorar la precisión en las búsquedas. Se recomienda mantener en producción tanto los embeddings de 512 dimensiones como versiones de mayor dimensionalidad para implementar estrategias de búsqueda simultánea o re-ranking.

El sistema no ha sido sometido a auditorías de seguridad ni cuenta con protecciones

específicas contra ataques informáticos, por lo que su uso debe limitarse estrictamente a entornos de investigación y desarrollo. Antes de cualquier despliegue en producción, resulta imprescindible realizar pruebas de penetración y aplicar medidas de seguridad apropiadas.

En caso de considerar la implementación de modelos de lenguaje de gran escala (LLMs) de manera local, se recomienda el uso de infraestructura con procesadores gráficos (GPUs) compatibles con frameworks de deep learning, con el fin de reducir los tiempos de inferencia y optimizar la experiencia del usuario.

Capítulo 7

Conclusiones

Este trabajo de diploma representó un desafío integral que abarcó múltiples etapas del ciclo de vida de un proyecto de ciencia de datos e inteligencia artificial. A lo largo de su desarrollo, se aplicaron técnicas de análisis exploratorio de datos, limpieza y preprocesamiento, ingeniería de características (feature engineering), generación de embeddings multimodales, entrenamiento de modelos de clasificación supervisada, implementación de modelos de lenguaje de gran escala con capacidades de visión, diseño de arquitecturas multiagente con orquestación, y desarrollo de una interfaz de usuario funcional. La integración de estos componentes permitió materializar los conocimientos adquiridos durante la Diplomatura en Ciencia de Datos e Inteligencia Artificial.

El proyecto facilitó la comprensión de las fases de conceptualización, diseño, implementación y evaluación de sistemas de inteligencia artificial aplicados a dominios específicos. Asimismo, consolidó capacidades de toma de decisiones basada en evidencia y pensamiento crítico, habilidades fundamentales para abordar proyectos de envergadura similar en contextos profesionales y académicos.

Bibliografía

- [1] Leo Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, August 1996. doi: 10.1023/A:1018054314350.
- [2] Giuseppe Cartella, Alberto Baldrati, Davide Morelli, Marcella Cornia, Marco Bertini, and Rita Cucchiara. OpenFashionCLIP: Vision-and-Language Contrastive Learning with Open-Source Fashion Data. In *Proceedings of the International Conference on Image Analysis and Processing*, 2023. URL: <https://arxiv.org/pdf/2309.05551>.
- [3] Facebook AI. Modelo de Categorización de Facebook. <https://huggingface.co/facebook/bart-large-mnli>, 2024. modelo tipo BART (Bidirectional and Auto-Regressive Transformer) entrenado por Facebook AI, pensado para la tarea de Reconocimiento de Interferencia Natural (NLI).
- [4] Wei-Lin Hsiao and Kristen Grauman. Creating capsule wardrobes from fashion images, 2017. URL: <https://arxiv.org/abs/1712.02662>, arXiv:1712.02662.
- [5] Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *arXiv preprint arXiv:1603.09320*, 2016. URL: <https://arxiv.org/abs/1603.09320>.
- [6] Microsoft Azure. Azure openai service. <https://azure.microsoft.com/en-us/products/cognitive-services/openai-service>, 2024. Facilitado por Styles Designer para este trabajo.
- [7] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. Catboost: unbiased boosting with categorical features. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31, pages 6638–6648, 2018. URL: <https://proceedings.neurips.cc/paper/2018/hash/14491b756b3a51daac41c24863285549-Abstract.html>.
- [8] Styles Designer. Startup con financiación ANII Uruguay. <https://stylesdesigner.com/>, 2024. Todos los datos de este trabajo son propiedad de Styles Designer.
- [9] University of Michigan. Ai-powered fashion: How tech is reshaping the future of zara’s fashion empire. <https://sites.lsa.umich.edu/mje/2025/04/04/ai-powered-fashion-how-tech-is-reshaping-the-future-of-zaras-fashion-empire/>, 2025. Artículo sobre IA en la moda.
- [10] Miglena Vasileva, Bryan A. Plummer, Karan Dusad, Rahul Rajpal, Shreya Kumar, and David Forsyth. Fashion outfit generation for e-commerce, 2019. URL: <https://arxiv.org/abs/1904.00741>, arXiv:1904.00741.
- [11] Ruizhongtai Zhao, Ke Cheng, and Jiashi Lu. Fashion recommendation and compatibility prediction using relational network, 2020. URL: <https://arxiv.org/abs/2005.06584>, arXiv:2005.06584.
- [12] Ruizhongtai Zhao, Min Wu, Ke Cheng, Yuncheng Jiang, and Jiashi Lu. Fashionnet: Personalized outfit recommendation with deep neural network, 2018. URL: <https://arxiv.org/abs/1810.02443>, arXiv:1810.02443.

- [13] Zilliz and Open-Source Contributors. pgvector: Open-source vector similarity search for postgresql. <https://github.com/pgvector/pgvector>, 2023. Extensión para almacenar y buscar embeddings mediante índices vectoriales como IVFFlat y HNSW.

Anexos

Los siguientes anexos complementan este documento y están disponibles como archivos separados:

ANEXO I: Limpieza de Datos

Archivo: ANEXO I LIMPIEZA DE DATOS.ipynb

Procedimientos de limpieza y normalización del dataset de productos de moda, incluyendo: eliminación de duplicados, corrección de valores nulos, estandarización de categorías y géneros, y validación de embeddings.

ANEXO II: Entrenamiento CatBoost con Categoría y Género de Tiendas

Archivo: ANEXO II ENTRENAMIENTO CATBOOST CON CATEGORÍA Y GÉNERO DE TIENDAS.ipynb

Proceso de entrenamiento del modelo CatBoost utilizando como features adicionales las etiquetas de categoría y género proporcionadas por las tiendas. Incluye: selección de hiperparámetros, métricas de validación.

ANEXO III: Entrenamiento CatBoost sin Categoría ni Género de Tiendas

Archivo: ANEXO III ENTRENAMIENTO CARBOOST SIN CATEGORÍA NI GÉNERO DE TIENDAS.ipynb

Experimento de entrenamiento del modelo CatBoost utilizando únicamente embeddings textuales (1024 dimensiones) sin incluir la categoría y el género proporcionado por la tienda. Documenta la reducción en precisión y las estrategias de compensación por peso en las variables objetivo.

ANEXO IV: Diagrama de Flujo Completo - Modo LLM Inteligente

Archivo: Anexo IV Diagrama de flujo con LLM.svg

Diagrama de flujo detallado del sistema multi-agente operando en modo inteligente con LLMs habilitados. Muestra: coordinación dinámica entre **7 agentes** (AnalizadorConsulta, ImageAnalyzer, ClasificadorCatBoost, BuscadorEmbeddings, PresentadorResultados, EspecialistaOutfits, RefinadorConsulta), flujos de decisión adaptativos (7 patrones identificados), y logging completo de interacciones LLM en base de datos.

ANEXO V: Diagrama de Flujo - Modo Fallback sin LLM

Archivo: ANEXO V Diagrama de flujo sin LLM.svg

Diagrama de flujo del sistema operando en modo fallback cuando los LLMs no están disponibles. Incluye: flujo estático predefinido (6 pasos obligatorios), búsqueda ML visual para inferir metadata, y generación de respuestas mediante templates predefinidos.

ANEXO VI: Modelo Entidad-Relación (MER) - Interfaz de Usuario

Archivo: ANEXO VI MER INTERFAZ DE USUARIO UI.svg

Esquema completo de la base de datos de la interfaz de usuario (11 tablas UI). Documenta: relaciones entre entidades, políticas de integridad referencial (CASCADE DELETE), índices optimizados, triggers automáticos, y vistas materializadas para analytics.